HỌC VIỆN CÔNG NGHỆ BƯU CHÍNH VIỄN THÔNG
KHOA ĐÀO TẠO SAU ĐẠI HỌC

BÁO CÁO BÀI TẬP LỚN

MÔN: TÌM KIẾM VÀ TRUY XUẤT THÔNG TIN

Đề tài: No More K-means: Single-Stage Sparse Coding for Efficient Multi-Vector Retrieval

Giảng viên hướng dẫn	:	**TS. Đào Thị Thúy Quỳnh**
Ngành	:	**Hệ thống thông tin**
Lớp	:	**M25CQHT02-B**
Học viên nhóm 9	:	**Nguyễn Văn Toàn - B25CHHT115**
	:	**Phạm Văn Thọ - B25CHHT114**
	:	**Hà Phương Nguyên - B25CHHT108**
	:	**Trần Duy Khánh - B25CHHT100**
	:	**Khamsing Outhaihueng - B25CHHT125**

Hà Nội - Tháng 10 năm 2026

PHÂN CÔNG NHIỆM VỤ NHÓM THỰC HIỆN

TT	Công việc / Nhiệm vụ	Học viên thực hiện
1	Điều phối, tổng hợp và biên tập báo cáo; Chương 1 (bối cảnh, động lực, nghiên cứu liên quan)	Nguyễn Văn Toàn - B25CHHT115
2	Dịch bài báo; Chương 2 (biểu diễn thưa, late interaction, cơ sở lý thuyết sai lệch bị chặn)	Phạm Văn Thọ - B25CHHT114
3	Dịch bài báo; Chương 3 (huấn luyện lai, các thành phần hàm loss, phân tích ablation)	Hà Phương Nguyên - B25CHHT108
4	Cài đặt môi trường, chạy lại mã nguồn và checkpoint, rà soát và sửa lỗi, tối ưu thuật toán truy hồi, đo so với ColBERTv2/PLAID; Chương 4 và mục 5.3–5.4	Trần Duy Khánh - B25CHHT100
5	Tổng hợp kết quả thực nghiệm của bài báo (mục 5.1–5.2), thảo luận và kết luận; làm slide trình bày	Khamsing Outhaihueng - B25CHHT125

NHÓM THỰC HIỆN TỰ ĐÁNH GIÁ

TT	Học viên	Thái độ tham gia	Mức hoàn thành CV	Kỹ năng giao tiếp	Kỹ năng hợp tác	Kỹ năng lãnh đạo
1	Nguyễn Văn Toàn - B25CHHT115	5	5	5	5	5
2	Phạm Văn Thọ - B25CHHT114	5	5	5	5	4
3	Hà Phương Nguyên - B25CHHT108	5	5	5	5	4
4	Trần Duy Khánh - B25CHHT100	5	5	5	5	4
5	Khamsing Outhaihueng - B25CHHT125	5	5	5	5	4

Ghi chú: thang điểm 0–5 cho từng tiêu chí (0: không tham gia / không hoàn thành; 5: chủ động, tích cực / hoàn thành xuất sắc).

TÓM TẮT

Multi-Vector Retrieval (MVR) mà ColBERT là đại diện đạt độ chính xác cao nhờ giữ biểu diễn ở mức token và so khớp bằng toán tử MaxSim, nhưng phải trả giá bằng chi phí lưu trữ và tính toán rất lớn. Các hệ thống hiện đại như ColBERTv2/PLAID khắc phục bằng cách giảm chiều và K-means clustering trên hàng tỷ vector token, dẫn tới thời gian lập chỉ mục kéo dài hàng trăm giờ và mất mát thông tin ngữ nghĩa do nén. Báo cáo này tìm hiểu bài báo *No More K-means* (ICML 2026) và phương pháp Single-stage Sparse Retrieval (SSR): mỗi embedding token được chiếu lên một không gian 16.384 chiều nhưng chỉ có 32 chiều khác không bằng TopK Sparse Autoencoder, nhờ đó mỗi neuron hoạt động đóng vai trò một "pseudo-token" và hệ thống dùng thẳng inverted index thay cho clustering.

Về lý thuyết, báo cáo trình bày (i) biểu diễn thưa và late interaction trên không gian thưa cùng cận sai lệch so với điểm dense, (ii) hàm mục tiêu huấn luyện lai giữa reconstruction và contrastive, (iii) cấu trúc inverted index mức neuron và chiến lược pruning từ thô đến tinh SSR++. Theo bài báo, SSR giảm thời gian lập chỉ mục khoảng 15 lần, giảm gần một nửa độ trễ truy vấn và đạt nDCG@10 trung bình 53,4 trên BEIR, cao hơn SPLADE-v3 (51,2) và PLAID (49,3).

Về thực nghiệm, nhóm đã chạy lại mã nguồn và checkpoint công bố, đối chiếu từng hàm với công thức trong bài, phát hiện và sửa mười điểm sai lệch (trong đó lỗi công thức chấm điểm trên inverted index làm nDCG@10 NFCorpus rơi từ 43,97 xuống 8,33 và chưa từng được báo cáo), đóng gói thành patch kèm unit test. Kiểm chứng trên NFCorpus, SciFact, FiQA, Quora và MS MARCO với ColBERTv2 làm đối chứng cho thấy checkpoint công bố chỉ vượt ColBERTv2 trên đúng split test của BEIR, còn trên split sạch và trên MS MARCO in-domain thì kém ColBERTv2 từ 0,7 đến 12,7 điểm. Phần cải tiến mã nguồn gồm kernel truy hồi exact mới `fast_hyb` cho kết quả xếp hạng giống hệt bản gốc nhưng nhanh hơn 30 lần trên FiQA (54 ms so với 1.635 ms mỗi truy vấn), dựng chỉ mục trong 4,8 s thay vì khoảng 280 s, một bản exact trên GPU, và phép xấp xỉ LB-rerank giữ 99,7% top-10 của phép tính exact với 0,67 lần thời gian, tốt hơn SSR++ của bài báo trên corpus lớn.

Từ khóa: Multi-Vector Retrieval, Sparse Autoencoder, inverted index, late interaction, ColBERT, pruning từ thô đến tinh, tái lập nghiên cứu.

MỤC LỤC

TÓM TẮT . . . . . **ii**

DANH MỤC CÁC TỪ VIẾT TẮT . . . . . **ix**

1 Background & Motivation **1**
1.1 Bài toán truy hồi thông tin 1
1.1.1 Định nghĩa hình thức 1
1.1.2 Yêu cầu của hệ thống truy hồi thực tế 1
1.2 Các hướng tiếp cận truy hồi 1
1.2.1 Lexical retrieval với BM25 và inverted index 1
1.2.2 Single-Vector Retrieval (SVR) 2
1.2.3 Multi-Vector Retrieval (MVR) và late interaction 2
1.3 Kiến trúc MVR hiện đại: pipeline ba tầng của ColBERTv2/PLAID 3
1.4 Hạn chế của hướng tiếp cận dựa trên K-means 3
1.5 Câu hỏi nghiên cứu, ý tưởng của SSR và thông tin bài báo 4
1.5.1 Câu hỏi nghiên cứu và ý tưởng cốt lõi 4
1.5.2 Thông tin xuất bản 4
1.6 Các nghiên cứu liên quan 5
1.7 Mục tiêu, phạm vi và phương pháp thực hiện của báo cáo 5
1.7.1 Mục tiêu 5
1.7.2 Phạm vi 5
1.7.3 Phương pháp 6
1.7.4 Cấu trúc báo cáo 6

2 Sparse Representation & Late Interaction **7**
2.1 Từ biểu diễn dense sang biểu diễn thưa 7
2.1.1 Hạn chế của biểu diễn dense ít chiều 7
2.1.2 Sparse coding và giả thuyết superposition 7
2.2 TopK Sparse Autoencoder (giới thiệu) 7
2.3 Late interaction trên không gian thưa 8
2.4 Hai biến thể SSR-tok và SSR-CLS 8
2.5 Cơ sở lý thuyết: tính chất sai lệch bị chặn 9
2.5.1 Thiết lập 9
2.5.2 Bốn giả thiết 9
2.5.3 Định lý A: sai lệch mức token 9
2.5.4 Định lý B: sai lệch mức MaxSim 10
2.5.5 Ý nghĩa và nhận xét 10

3 Hybrid Training of Sparse Projectors **11**
3.1 Yêu cầu đối với sparse projector 11
3.2 TopK Sparse Autoencoder 11

3.3 Hàm loss không giám sát . . . 12
3.3.1 Reconstruction loss $\mathcal{L}_{\text{recon}}(k)$ . . . 12
3.3.2 Multi-TopK: số hạng $\frac{1}{8}\mathcal{L}_{\text{recon}}(4k)$ . . . 12
3.3.3 Auxiliary loss $\mathcal{L}_{\text{aux}}(k_{\text{aux}})$ chống neuron chết . . . 12
3.3.4 Sparse contrastive loss $\mathcal{L}_{\text{cl}}$ . . . 12
3.4 Supervised contrastive loss $\mathcal{L}_{\text{CE}}$ . . . 13
3.5 Hàm mục tiêu tổng . . . 13
3.6 Phân tích thực nghiệm vai trò của từng hàm loss . . . 13
3.7 Đối chiếu thiết lập huấn luyện với mã nguồn công bố . . . 14

4 Indexing & Retrieval: SSR và SSR++ . . . 16
4.1 Nhắc lại inverted index trong lexical retrieval . . . 16
4.2 Inverted index mức neuron . . . 16
4.3 Truy hồi một giai đoạn (SSR) . . . 17
4.4 Truy hồi tăng tốc SSR++ (từ thô đến tinh) . . . 17
4.4.1 Bước 1: chấm điểm thô . . . 17
4.4.2 Bước 2: tinh chỉnh chính xác . . . 17
4.4.3 Nhận xét về tính xấp xỉ của bước thô . . . 18
4.5 Phân tích độ phức tạp . . . 18
4.6 Hiệu quả của SSR++ theo bài báo . . . 19
4.7 Những điểm cần chú ý khi cài đặt . . . 19

5 Experiments, Code Improvement & Conclusion . . . 20
5.1 Thiết lập thực nghiệm của bài báo . . . 20
5.1.1 Bộ dữ liệu . . . 20
5.1.2 Độ đo . . . 20
5.1.3 Phương pháp so sánh và cấu hình . . . 21
5.2 Kết quả của bài báo . . . 21
5.2.1 Hiệu năng trong điều kiện có kiểm soát . . . 21
5.2.2 Khả năng mở rộng với backbone LLM . . . 22
5.2.3 Các đánh giá bổ sung: LoTTE, văn bản dài và LIMIT . . . 22
5.2.4 Hiệu quả tính toán và tài nguyên hệ thống . . . 22
5.2.5 Phân tích siêu tham số h và K . . . 24
5.2.6 Các phiên bản của bài báo và vấn đề độ đo . . . 25
5.3 Cài đặt, kiểm chứng và tái lập của nhóm . . . 25
5.3.1 Môi trường và dữ liệu . . . 25
5.3.2 Cấu trúc mã nguồn công bố . . . 25
5.3.3 Kiểm tra pipeline đánh giá bằng baseline . . . 26
5.3.4 Rà soát mã nguồn: các lỗi phát hiện và cách sửa . . . 26
5.3.5 Kết quả chạy lại trên split test . . . 27
5.3.6 Kiểm tra theo split: điểm chỉ vọt lên ở split test . . . 28
5.3.7 Đánh giá in-domain trên MS MARCO . . . 29
5.3.8 So sánh hiệu năng với ColBERTv2 + PLAID . . . 29
5.3.9 Thông tin từ trả lời của tác giả trên GitHub . . . 30
5.3.10 Giới hạn của thực nghiệm . . . 30
5.4 Cải tiến mã nguồn: tối ưu thuật toán truy hồi . . . 30
5.4.1 Vị trí tối ưu trong pipeline truy hồi . . . 31
5.4.2 Kernel exact mới `fast_hyb` . . . 31
5.4.3 Kernel exact trên GPU . . . 32

5.4.4 Phương án xấp xỉ mới: LB-rerank . . . 33
5.4.5 Các hướng đã thử nhưng không hiệu quả . . . 33
5.4.6 Tổng kết phần cải tiến . . . 33
5.5 Thảo luận . . . 34
5.5.1 Điểm mạnh của phương pháp SSR . . . 34
5.5.2 Hạn chế của bài báo và mô hình . . . 34
5.5.3 Khả năng ứng dụng thực tiễn . . . 34
5.5.4 Tác động xã hội . . . 35
5.6 Kết luận và hướng phát triển . . . 35
5.6.1 Kết luận . . . 35
5.6.2 Hướng phát triển . . . 35

TÀI LIỆU THAM KHẢO . . . **37**

A Chi tiết các lỗi trong mã nguồn và bản sửa . . . **39**
A.1 Lỗi 2: nạp checkpoint tạo SAE ngẫu nhiên . . . 39
A.2 Lỗi 1: công thức chấm điểm trên inverted index . . . 39
A.3 Lỗi 3 và 4: ánh xạ neuron và bước thô SSR++ . . . 40
A.4 Lỗi 5–7 và 9: độ đo và huấn luyện . . . 40
A.5 Lỗi 8: dựng chỉ mục với corpus lớn . . . 41

B Hướng dẫn chạy lại thí nghiệm . . . **42**

DANH MỤC CÁC HÌNH ẢNH

1.1 Hai cách biểu diễn tài liệu: SVR nén thành một vector rồi lấy tích vô hướng; MVR giữ một vector cho mỗi token rồi lấy MaxSim theo từng token truy vấn và cộng lại. 2
1.2 So sánh pipeline của ColBERTv2/PLAID (ba tầng truy hồi, cần K-means khi lập chỉ mục) và SSR (một giai đoạn trên inverted index). Vẽ lại theo Hình 2 của bài báo [1]. 3
1.3 Chất lượng trung bình trên BEIR so với độ trễ truy vấn của các phương pháp, vẽ lại từ Bảng 1 của bài báo (bản arXiv v3; xem Bảng 5.2). Theo số liệu công bố, hai biến thể SSR nằm ở góc trên bên trái, tức vừa nhanh vừa chính xác hơn các baseline. 4

2.1 Luồng biến đổi một token trong SSR: backbone tạo dense embedding, SAE TopK tạo sparse code 16.384 chiều với 32 giá trị khác không, mỗi chiều hoạt động trở thành một pseudo-token và được ghi vào inverted index. 7
2.2 Phân rã sai số thành hai thành phần: sai số tái tạo của SAE ($\leq 2B\epsilon + \epsilon^2$) và sai số hình học của decoder ($\leq \delta C^2$). 10

3.1 Tổng quan quá trình huấn luyện SAE trong SSR: embedding token từ backbone đi qua encoder và toán tử TopK để tạo sparse code, sau đó được giải mã; hàm mục tiêu gồm bốn nhóm thành phần: tái tạo, chống neuron chết, contrastive giữa các token và contrastive giữa truy vấn và tài liệu. . . . 11
3.2 Đóng góp tích lũy của từng thành phần hàm loss cho SSR-tok và SSR-CLS (vẽ lại từ số liệu Bảng 3.3). 13
3.3 Độ nhạy của chất lượng theo ba trọng số α, β, γ (vẽ lại từ Bảng 3.4). 14

4.1 Cấu trúc inverted index mức neuron: mỗi tài liệu đóng góp giá trị $\mu_{D,u}$ vào posting list của từng neuron mà nó kích hoạt; mỗi posting list chia thành block và mỗi block lưu cận trên U_B. 17
4.2 Sơ đồ SSR++: bước chấm điểm thô với 4 neuron mỗi token và pruning theo cận trên của block, rồi bước tinh chỉnh với đủ 32 neuron trên 2.000 ứng viên. 18

5.1 nDCG@10 trung bình trên BEIR của các phương pháp tiêu biểu theo bài báo (vẽ lại từ Bảng 5.2). 21
5.2 Recall@5 trên benchmark LIMIT theo bài báo (vẽ lại từ Bảng 5.5): khoảng cách rất lớn giữa các mô hình SVR và MVR. 23
5.3 Thời gian lập chỉ mục MS MARCO trên CPU của các phương pháp MVR theo bài báo (vẽ lại từ Bảng 5.6). 23
5.4 Trái: thời gian huấn luyện, lập chỉ mục và truy hồi trong toàn bộ pipeline. Phải: chất lượng và độ trễ so với ColBERTv2 ở các quy mô dữ liệu khác nhau. Nguồn: Hình 3 của bài báo [1]. 23
5.5 Trái: ảnh hưởng của hidden dimension h (từ 2^{12} đến 2^{16}). Phải: ảnh hưởng của mức thưa K (từ 8 đến 128). Nguồn: Hình 4 của bài báo [1]. 24
5.6 Ảnh hưởng của K lên chất lượng theo từng nhóm miền dữ liệu (vẽ lại từ Bảng 5.7); đường đứt đánh dấu $K = 32$. 24
5.7 Chất lượng trên split sạch và split test (nhóm đo). Chỉ SSR-tok (có SAE) có bước nhảy lớn ở split test. 28
5.8 Pipeline truy hồi của SSR và vị trí các tối ưu của nhóm. Hàng trên: dựng chỉ mục; hàng giữa: truy hồi exact (công thức (4)); hàng dưới: đường xấp xỉ, trong đó bước thô của SSR++ được thay bằng LB-rerank. 31
5.9 Độ trễ truy vấn trên FiQA (nhóm đo). Mọi phương án của SSR trừ SSR++ đều cho kết quả xếp hạng giống hệt bản gốc. 32

DANH MỤC CÁC BẢNG

1.1 So sánh ba họ phương pháp truy hồi: BM25, SVR và MVR (ColBERT). 2
1.2 Tổng hợp hạn chế của phương pháp dựa trên K-means và cách SSR giải quyết, làm cầu nối sang các chương sau. 4

2.1 So sánh biểu diễn dense của ColBERT và biểu diễn thưa của SSR. 7
2.2 Ví dụ tính tay điểm thưa cho token truy vấn q_1 với hai token tài liệu. 8
2.3 Các giả thiết của phân tích sai lệch bị chặn. 9

3.1 Siêu tham số huấn luyện SAE với backbone BERT-base theo Bảng 6 của bài báo. 12
3.2 Các thành phần của hàm loss và vai trò của từng thành phần. 13
3.3 Ablation từng thành phần của hàm loss theo bài báo (nDCG@10 trung bình BEIR, $K = 32$). . . . 13
3.4 Độ nhạy của nDCG@10 trung bình BEIR theo ba trọng số α, β, γ (theo bài báo). 14
3.5 Thiết lập huấn luyện: bài báo, mặc định của mã nguồn công bố, và trả lời của tác giả trong issue. Cột cuối là cách xử lý của nhóm. 15

4.1 So sánh chi phí lập chỉ mục trên MS MARCO theo bài báo. 17
4.2 So sánh độ phức tạp giữa PLAID, SSR và SSR++. 19
4.3 So sánh SSR và SSR++ trên MS MARCO passage theo bài báo (Bảng 5). 19

5.1 Các bộ dữ liệu dùng trong đánh giá của bài báo. 20
5.2 Bảng 1 của bài báo (bản arXiv v3): nDCG@10 trên 13 tập BEIR, cột MS là MS MARCO (xem mục 5.2.6 về độ đo của cột này), độ trễ ms/truy vấn trên MS MARCO. In đậm: tốt nhất; gạch chân: thứ hai. 21
5.3 Kết quả theo bài báo khi dùng Llama-embed-nemotron-8b làm backbone đóng băng (trung bình nDCG@10 trên 14 tập). 22
5.4 Success@5 trên LoTTE theo bài báo cho hai loại truy vấn Search và Forum. 22
5.5 Kết quả trên bộ LIMIT theo bài báo. 22
5.6 So sánh hiệu quả trên CPU theo bài báo (MS MARCO passage; chỉ mục tính bằng giờ, truy hồi tính bằng ms/truy vấn). 23
5.7 nDCG@10 theo mức thưa K và theo nhóm miền dữ liệu (theo bài báo). 24
5.8 K cố định và K thích ứng theo độ dài truy vấn (theo bài báo). 25
5.9 Môi trường thực nghiệm của nhóm. 25
5.10 Các tập dữ liệu nhóm đã dùng. 26
5.11 Tái lập số đã công bố của các baseline bằng pipeline của nhóm. 26
5.12 Các sai lệch giữa mã nguồn công bố và bài báo. 27
5.13 nDCG@10 trên split test (nhóm đo). “Repo gốc” = checkpoint công bố + mã gốc (đã sửa lỗi nạp SAE); “đã sửa” = MaxSim đúng công thức (4); SSR++ theo cấu hình Phụ lục D.1. 27
5.14 nDCG@10 trên split sạch (train/dev) và split test của cùng một tập (nhóm đo). Quora dùng quy ước BEIR bỏ tài liệu trùng id với query. 28
5.15 Re-ranking BM25 top-100 trên MS MARCO (nhóm đo). Pipeline SSR được kiểm tra lại: sparse code của query trùng 235/235 dòng với hàm của repo, điểm khớp kernel exact tới $1{,}4 \cdot 10^{-6}$. . . . 29

5.16 Hiệu năng trên FiQA (nhóm đo). Thời gian mã hóa corpus không tính vào "dựng chỉ mục" (ColBERTv2 ≈260 s, SSR ≈420 s trên GPU). 29
5.17 Đối chiếu từng tuyên bố về hiệu năng của bài báo với số nhóm đo trên FiQA. 30
5.18 Thông tin từ trả lời của tác giả và đối chiếu của nhóm. 30
5.19 Từng bước của pipeline: cách làm của bản gốc (repo đã sửa) và của bản tối ưu. 31
5.20 Độ trễ của các kernel exact (nhóm đo; kết quả xếp hạng giống hệt bản gốc). 32
5.21 Các phương án xấp xỉ trên FiQA dev (nhóm đo, 6 luồng; exact = 82 ms, nDCG@10 22,80). 33
5.22 Độ chặt của cận trên dùng để pruning (nhóm đo trên 40 query mỗi tập). θ: ngưỡng top-100 trung bình mỗi token query; "UB $\geq \theta$": tỉ lệ tài liệu có cận trên vượt ngưỡng (không bỏ qua được). . . . 33

DANH MỤC CÁC TỪ VIẾT TẮT

Viết tắt	**Cụm từ tiếng Anh**	**Giải thích**
ANN	Approximate Nearest Neighbor	Tìm láng giềng gần đúng
BEIR	Benchmarking IR	Bộ 18 tập dữ liệu đánh giá truy hồi zero-shot
BM25	Best Matching 25	Hàm xếp hạng từ vựng cổ điển
CE	Cross-Entropy	Mất mát entropy chéo
CLS	Classification token	Token đặc biệt đứng đầu chuỗi của BERT
IR	Information Retrieval	Truy xuất thông tin
MaxSim	Maximum Similarity	Phép tương tác muộn của ColBERT
MRR	Mean Reciprocal Rank	Trung bình nghịch đảo thứ hạng
MVR	Multi-Vector Retrieval	Truy hồi đa vector (mỗi tài liệu là nhiều vector token)
nDCG	normalized Discounted Cumulative Gain	Độ lợi tích lũy chiết khấu chuẩn hóa
PLAID	Performance-optimized Late Interaction Driver	Bộ máy truy hồi của ColBERTv2 dùng K-means và nén residual
SAE	Sparse Autoencoder	Bộ tự mã hóa thưa
SSR	Single-stage Sparse Retrieval	Truy hồi thưa một giai đoạn (phương pháp của bài báo)
SSR++	SSR with coarse-to-fine pruning	SSR có thêm bước lọc thô rồi chấm tinh
SVR	Single-Vector Retrieval	Truy hồi đơn vector
TopK	Top-K activation	Chỉ giữ K giá trị lớn nhất, còn lại đặt bằng 0
UB	Upper Bound	Cận trên (của một khối posting)
VRAM	Video RAM	Bộ nhớ của GPU

CHƯƠNG 1. BACKGROUND & MOTIVATION

Chương mở đầu trình bày bối cảnh của bài toán truy hồi thông tin, đối chiếu ba họ phương pháp (lexical retrieval, Single-Vector Retrieval và Multi-Vector Retrieval) và chỉ ra vì sao bước K-means clustering trong các hệ MVR hiện đại trở thành nút thắt. Từ đó, chương giới thiệu ý tưởng của SSR, thông tin về bài báo, các nghiên cứu liên quan, rồi nêu mục tiêu, phạm vi và phương pháp thực hiện của báo cáo.

1.1. Bài toán truy hồi thông tin

1.1.1. Định nghĩa hình thức

Cho một truy vấn Q và một kho tài liệu lớn $\mathcal{D} = \{D_1, \ldots, D_{|\mathcal{D}|}\}$, trong đó cả Q lẫn mỗi tài liệu D đều là chuỗi token. Nhiệm vụ của hệ truy hồi là trả về danh sách top-k tài liệu được xếp hạng theo mức độ liên quan đến Q. Cách tiếp cận học máy phổ biến là học một hàm chấm điểm có tham số $s_\theta(Q, D) \in \mathbb{R}$ sao cho với mọi cặp tài liệu liên quan D^+ và không liên quan D^-:

$$s_\theta(Q, D^+) > s_\theta(Q, D^-). \tag{1.1}$$

Trong thực tế, s_θ được ước lượng từ tập huấn luyện gồm các bộ ba (Q, D^+, D^-), và chất lượng xếp hạng được đo bằng các độ đo như MRR@10 hay nDCG@10 (định nghĩa chính xác ở mục 5.1.2). Khác biệt giữa các họ phương pháp nằm ở chỗ Q và D được biểu diễn như thế nào.

1.1.2. Yêu cầu của hệ thống truy hồi thực tế

Một hệ truy hồi dùng trong sản phẩm không chỉ cần đúng mà còn phải nhanh và rẻ. Ba tiêu chí sau luôn đánh đổi lẫn nhau:

- **Độ chính xác:** đo bằng MRR@10 và nDCG@10 trên các benchmark như MS MARCO hoặc BEIR.
- **Độ trễ truy vấn:** thời gian trả kết quả cho một truy vấn, tính bằng mili-giây.
- **Chi phí lập chỉ mục và lưu trữ:** số giờ xây chỉ mục trên toàn kho, bộ nhớ đỉnh, kích thước chỉ mục, và khả năng thêm tài liệu mới mà không phải dựng lại từ đầu.

Hai độ đo chất lượng được dùng lặp lại trong suốt báo cáo. MRR@10 là nghịch đảo thứ hạng của tài liệu liên quan đầu tiên trong mười kết quả đầu, bằng không nếu không có tài liệu liên quan nào trong top-10. nDCG@10 cộng dồn độ liên quan có chiết khấu theo vị trí trong top-10 rồi chuẩn hóa theo xếp hạng lý tưởng. MRR@10 chỉ quan tâm kết quả đúng đầu tiên, còn nDCG@10 xét toàn bộ thứ tự xếp hạng, nên hai độ đo này không so sánh được với nhau; điều này sẽ quan trọng khi đọc các bảng của bài báo (mục 5.2.6).

1.2. Các hướng tiếp cận truy hồi

1.2.1. Lexical retrieval với BM25 và inverted index

Lexical retrieval so khớp từ một cách chính xác. Mỗi từ t trong kho được gắn với một *posting list* $I_t = \{(D, w_{D,t})\}$ liệt kê các tài liệu chứa nó cùng trọng số; cấu trúc dữ liệu này gọi là *inverted index*. BM25 [15] chấm điểm tài liệu bằng tổng có trọng số của tần suất từ, có chuẩn hóa theo độ dài tài liệu và độ hiếm của từ. Ưu điểm lớn nhất là tốc độ: truy vấn chỉ cần duyệt $|Q|$ posting list và cộng dồn điểm, chi phí tỉ lệ với tổng độ dài các posting chứ không với kích thước kho. Inverted index còn hỗ trợ safe pruning (WAND, block-max WAND): mỗi block posting lưu cận trên

của trọng số để bỏ qua những block không thể lọt vào top-k. Nhược điểm là không hiểu đồng nghĩa: hai câu diễn đạt cùng một ý bằng từ khác nhau sẽ không khớp. Inverted index cũng là hạ tầng mà SSR tái sử dụng ở Chương 4.

1.2.2. Single-Vector Retrieval (SVR)

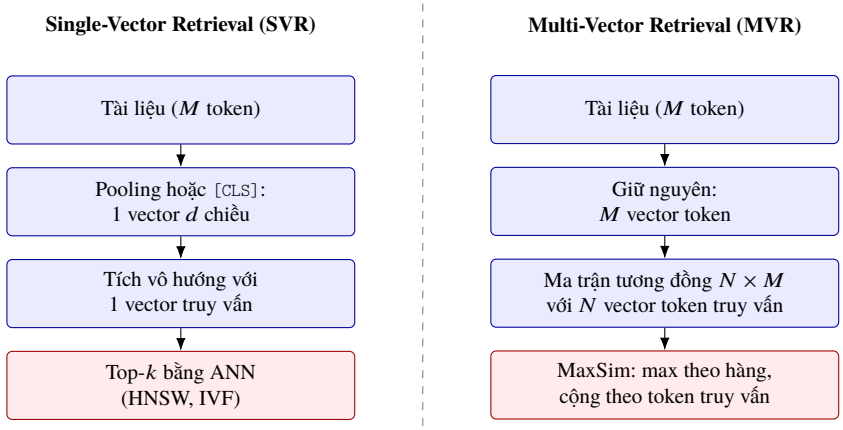
SVR nén toàn bộ tài liệu thành một vector duy nhất, thường lấy từ token `[CLS]` hoặc bằng pooling, rồi tính điểm bằng tích vô hướng giữa vector truy vấn và vector tài liệu. Tìm kiếm được thực hiện bằng các thuật toán approximate nearest neighbor (ANN) như FAISS, HNSW hay IVF. Các mô hình đại diện gồm DPR, e5-mistral, Qwen3-Embedding và Llama-Embed-8B. Hạn chế căn bản của SVR là nén cả văn bản vào một vector kích thước cố định, làm mất chi tiết ở mức token. Benchmark LIMIT [14] cho thấy rõ điều này: nhiều mô hình SVR mạnh chỉ đạt Recall@5 dưới 5% (Qwen3-4B đạt 1,2%; GritLM-7B đạt 4,7%), trong khi các mô hình MVR đạt trên 70% (Bảng 5.5).

1.2.3. Multi-Vector Retrieval (MVR) và late interaction

MVR giữ nguyên một embedding cho mỗi token của cả truy vấn lẫn tài liệu. ColBERT [2] mã hóa query thành $\{q_1, \ldots, q_N\}$ và tài liệu thành $\{d_1, \ldots, d_M\}$ (mỗi token một vector 128 chiều đã chuẩn hóa) rồi tính điểm bằng toán tử MaxSim: với mỗi token truy vấn q_i, tìm token tài liệu d_j tương đồng nhất, rồi cộng các cực đại này trên toàn bộ token truy vấn:

$$S(Q, D) = \sum_{i=1}^{N} \max_{j=1}^{M} q_i \cdot d_j. \tag{1.2}$$

Vì tương tác giữa hai chuỗi token chỉ diễn ra ở bước cuối sau khi đã mã hóa độc lập, cách tiếp cận này được gọi là *late interaction*. MaxSim cho phép so khớp từng khía cạnh ngữ nghĩa của truy vấn, đổi lại số vector phải lưu tăng hàng chục đến hàng trăm lần so với SVR (MS MARCO: khoảng 600 triệu vector token), và chấm điểm có độ phức tạp $O(N \times M)$ cho mỗi cặp. Hình 1.1 minh họa hai cách biểu diễn, Bảng 1.1 so sánh ba họ phương pháp.



Hình 1.1: Hai cách biểu diễn tài liệu: SVR nén thành một vector rồi lấy tích vô hướng; MVR giữ một vector cho mỗi token rồi lấy MaxSim theo từng token truy vấn và cộng lại.

Bảng 1.1: So sánh ba họ phương pháp truy hồi: BM25, SVR và MVR (ColBERT).

Tiêu chí	BM25	SVR	MVR (ColBERT)
Đơn vị biểu diễn	Từ	1 vector / tài liệu	1 vector / token
Hàm điểm	TF-IDF có chuẩn hóa	Tích vô hướng	MaxSim
Cấu trúc tìm kiếm	Inverted index	ANN (HNSW, IVF)	Clustering + nhiều tầng pruning
Hiểu ngữ nghĩa	Kém	Tốt (mức toàn cục)	Rất tốt (mức token)
Tốc độ	Rất nhanh	Nhanh	Chậm hơn
Kích thước chỉ mục	Nhỏ	Nhỏ	Rất lớn

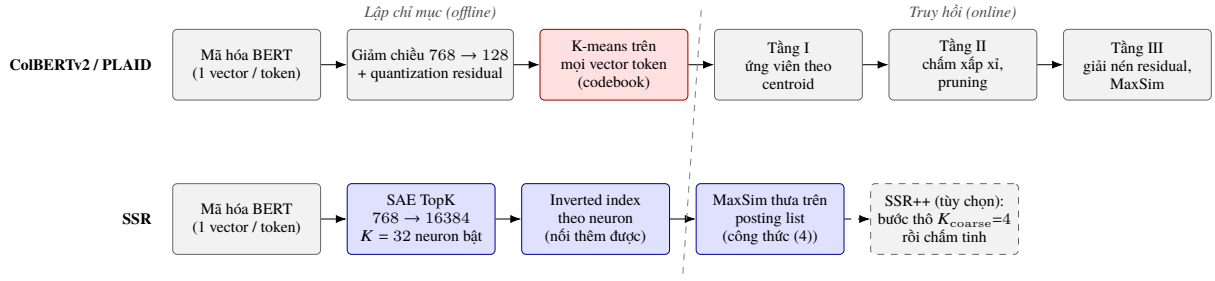
1.3. Kiến trúc MVR hiện đại: pipeline ba tầng của ColBERTv2/PLAID

Để giảm chi phí của MVR, ColBERTv2 [3] giảm số chiều embedding token (từ 768 xuống 128) rồi clustering bằng K-means với $\sqrt{16 \cdot \text{số token}}$ cụm; mỗi token được lưu dưới dạng chỉ số centroid gần nhất kèm residual quantization 1–2 bit. PLAID [4] tổ chức tìm kiếm thành ba tầng (Hình 1.2):

1. **Tầng I: sinh ứng viên.** Từ mỗi token truy vấn, lấy các centroid gần nhất C_i rồi gom tất cả tài liệu có token thuộc các cụm đó:
$$\mathcal{D}_{\text{cand}} = \bigcup_{q_i \in Q} \bigcup_{c \in C_i} \phi(c), \tag{1.3}$$
trong đó $\phi(c)$ là tập tài liệu có token thuộc centroid c. Tầng này loại bỏ hơn 98% kho.

2. **Tầng II: chấm điểm xấp xỉ và pruning.** Dùng centroid thay cho vector thật để ước lượng điểm, giữ lại vài nghìn tài liệu:
$$\hat{S}(Q, D) = \sum_{i=1}^{|Q|} \max_{j=1}^{|D|} q_i \cdot c_{d_j}. \tag{1.4}$$

3. **Tầng III: giải nén và xếp hạng lại.** Khôi phục vector xấp xỉ $\tilde{d}_j \approx c_{d_j} + r_{d_j}$ rồi tính MaxSim đầy đủ trên tập ứng viên nhỏ:
$$S(Q, D) = \sum_{i=1}^{N} \max_{j=1}^{M} q_i \cdot \tilde{d}_j. \tag{1.5}$$



Hình 1.2: So sánh pipeline của ColBERTv2/PLAID (ba tầng truy hồi, cần K-means khi lập chỉ mục) và SSR (một giai đoạn trên inverted index). Vẽ lại theo Hình 2 của bài báo [1].

1.4. Hạn chế của hướng tiếp cận dựa trên K-means

Cách tiếp cận trên có ba hạn chế chính, với số liệu do bài báo đưa ra:

- **Chi phí lập chỉ mục lớn và không cập nhật gia tăng được.** K-means phải chạy trên hàng tỷ vector token, sau đó còn tính residual cho từng token. Trên MS MARCO, ColBERTv2/PLAID mất khoảng 122,9 giờ lập chỉ mục, XTR/WARP khoảng 103,7 giờ, với bộ nhớ đỉnh lần lượt 274,2 GB và 186,7 GB. Khi có tài liệu mới, K-means không thể chèn thêm điểm dữ liệu một cách rẻ, buộc hệ thống phải xây lại chỉ mục.
- **Mất thông tin do nén.** Gom token về centroid cùng mã ngắn loại bỏ đúng phần chi tiết mức token mà MVR muốn giữ, khiến chất lượng xếp hạng bị chặn trần.
- **Pipeline nhiều tầng phức tạp.** Logic điều khiển rối, các lần truy cập bộ nhớ không liên tục và nhiều lượt giải nén làm tăng độ trễ. Bản thân MaxSim vẫn có độ phức tạp $O(N \times M)$, nên độ trễ tăng mạnh với tài liệu dài (ColBERTv2 lên 79,3 ms trên MS MARCO Document).

Bảng 1.2: Tổng hợp hạn chế của phương pháp dựa trên K-means và cách SSR giải quyết, làm cầu nối sang các chương sau.

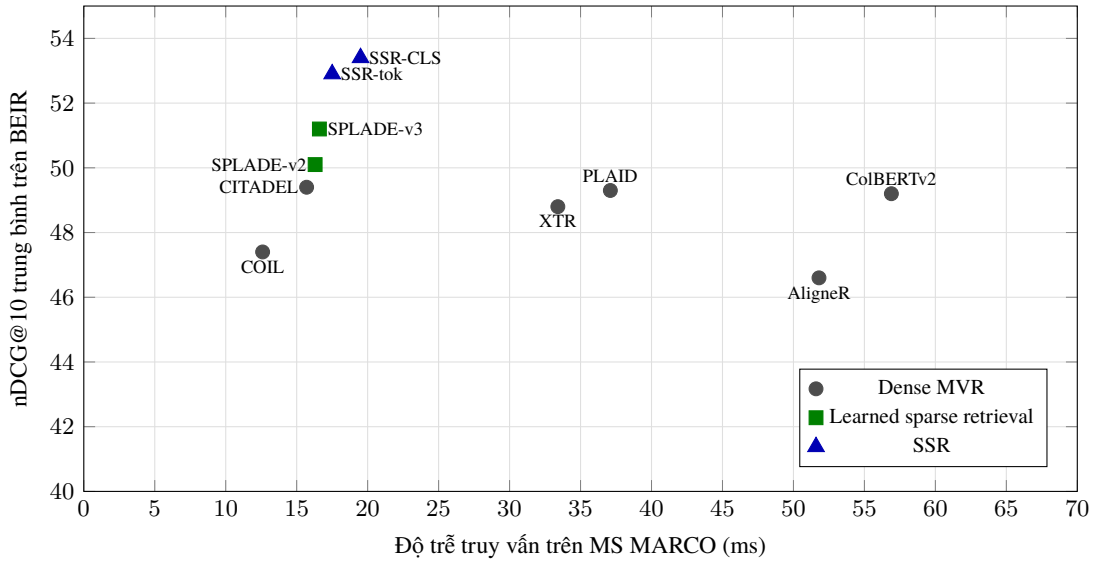
Hạn chế	Nguyên nhân	SSR giải quyết bằng	Trình bày ở
Lập chỉ mục chậm, phải xây lại	K-means trên hàng tỷ vector	Inverted index mức neuron, chỉ nối thêm	Chương 4
Mất thông tin	Nén về centroid, giảm chiều	Chiếu lên không gian nhiều chiều thưa	Chương 2
Pipeline phức tạp, $O(N \times M)$	Ba tầng cộng nhiều lượt giải nén	Một giai đoạn; SSR++ pruning từ thô đến tinh	Chương 4
Đặc trưng thưa phải đủ phân biệt	Sparse code có thể trùng nhau	Loss lai reconstruction + contrastive	Chương 3

1.5. Câu hỏi nghiên cứu, ý tưởng của SSR và thông tin bài báo

1.5.1. Câu hỏi nghiên cứu và ý tưởng cốt lõi

Từ các hạn chế trên, bài báo đặt câu hỏi: *liệu có thể giữ được độ chi tiết mức token của MVR, đạt tốc độ tương đương tìm kiếm từ vựng, mà không cần đến K-means clustering hay không?* Ý tưởng của SSR là đảo ngược hướng biến đổi không gian: thay vì nén embedding token xuống không gian ít chiều, SSR chiếu nó lên một không gian *nhiều chiều hơn nhưng cực kỳ thưa* nhờ Sparse Autoencoder (SAE), trong đó mỗi token chỉ kích hoạt $K = 32$ trên $h = 16.384$ neuron. Mỗi neuron hoạt động được coi như một "pseudo-token"; vì số neuron bật rất ít so với số chiều, mỗi neuron chỉ xuất hiện ở một phần nhỏ token, nên toàn bộ hạ tầng inverted index vốn dùng cho BM25 có thể được tái sử dụng.

Bài báo đóng góp ba điểm: (i) đề xuất chiếu embedding token lên không gian thưa nhiều chiều để thay thế clustering, cho phép truy hồi ngữ nghĩa đa vector qua một giai đoạn trên inverted index; (ii) thiết kế hàm mục tiêu huấn luyện lai kết hợp reconstruction và contrastive nhằm vừa giữ thông tin vừa bảo đảm khả năng phân biệt; (iii) xây dựng inverted index mức neuron cùng thuật toán truy hồi hai bước SSR++, kèm phân tích lý thuyết về sai lệch bị chặn so với điểm dense và đánh giá rộng trên MS MARCO, BEIR và backbone LLM 8B. Bài còn đề xuất hai biến thể: SSR-tok chỉ dùng điểm token thưa và SSR-CLS bổ sung tín hiệu ngữ nghĩa toàn cục từ token `[CLS]`. Hình 1.3 vẽ lại Bảng 1 của bài dưới dạng chất lượng so với độ trễ; các số liệu này được kiểm chứng ở Chương 5.



Hình 1.3: Chất lượng trung bình trên BEIR so với độ trễ truy vấn của các phương pháp, vẽ lại từ Bảng 1 của bài báo (bản arXiv v3; xem Bảng 5.2). Theo số liệu công bố, hai biến thể SSR nằm ở góc trên bên trái, tức vừa nhanh vừa chính xác hơn các baseline.

1.5.2. Thông tin xuất bản

- **Tên bài báo:** No More K-means: Single-Stage Sparse Coding for Efficient Multi-Vector Retrieval.

- **Tác giả:** Lixuan Guo*, Yifei Wang*, Tiansheng Wen, Aosong Feng, Stefanie Jegelka, Chenyu You (* đóng góp ngang nhau); Stony Brook University, Xidian University, Amazon AGI SF Lab, Georgia Tech, Yale University, TUM, MIT.
- **Nơi công bố:** Proceedings of the 43rd International Conference on Machine Learning (ICML 2026), PMLR 306 [21]. Bản tiền ấn phẩm arXiv:2605.30120 (v3 ngày 03/06/2026, v4 ngày 28/09/2026).
- **Mã nguồn và checkpoint:** https://github.com/Y-Research-SBU/SSR (giấy phép MIT, commit `c64bc05`) [19]; checkpoint `SSR-tok` tại https://huggingface.co/Y-Research-Group/SSR [20].
- **Lĩnh vực:** Information Retrieval, Multi-Vector Retrieval, Sparse Representation Learning.

1.6. Các nghiên cứu liên quan

Single-Vector Retrieval thời đại LLM. Các mô hình như Llama-Embed-8B hay Qwen3-Embedding cho thấy biểu diễn dense cấp văn bản vẫn rất mạnh nhờ tiền huấn luyện quy mô lớn. Nhiều kỹ thuật nén như giảm chiều theo từng mức (Matryoshka) hay quantization đã được áp dụng, và họ phương pháp thưa học được như SPLADE [9] cạnh tranh trực tiếp: SPLADE học trọng số thưa trên từ điển 30.522 từ của BERT và truy hồi bằng inverted index, nhưng gộp cả văn bản thành *một* vector thưa và không gian thưa là từ điển cố định.

Tối ưu hóa MVR. ColBERTv2 giảm chiều và nén residual [3]; PLAID [4] và EMVB [8] cải thiện khâu pruning nhiều tầng bằng centroid và bit-vector/SIMD; COIL và CITADEL quay lại inverted index nhờ token có ích cho khớp từ hoặc bộ định tuyến từ vựng; XTR và WARP đặt mục tiêu truy hồi token rẻ hơn; IGP, MUVERA và DESSERT tái tạo late interaction bằng các cấu trúc thay thế (đồ thị lân cận, mã hóa chiều cố định, băm).

Sparse Autoencoder (SAE). SAE được dùng nhiều trong nghiên cứu diễn giải mô hình ngôn ngữ, với các biến thể TopK [10], BatchTopK, JumpReLU và Gated. Tuy nhiên, chúng chủ yếu phục vụ phân tích cơ chế mô hình chứ chưa được dùng để thay thế clustering trong MVR.

Khoảng trống và định vị của SSR. SSR nằm ở giao điểm của hai dòng nghiên cứu: giữ tương tác giữa các token của ColBERT nhưng biểu diễn thưa học được từ SAE, không ràng buộc vào từ điển. SSR khác SPLADE và COIL ở chỗ: SPLADE thưa trên từ vựng ở mức văn bản; COIL dùng inverted index nhưng theo từ khớp chính xác; còn SSR thưa trên một không gian học được ở mức token, mỗi chiều là một neuron khái niệm chứ không phải một từ. Đây cũng là một trong những ứng dụng đầu tiên của TopK SAE vào bài toán truy hồi.

1.7. Mục tiêu, phạm vi và phương pháp thực hiện của báo cáo

1.7.1. Mục tiêu

- Trình bày cơ sở lý thuyết của Multi-Vector Retrieval, PLAID, TopK Sparse Autoencoder, late interaction trên không gian thưa, hàm mục tiêu huấn luyện lai, và thuật toán SSR/SSR++ của bài báo.
- Chạy lại mã nguồn và checkpoint do tác giả công bố; đối chiếu mã nguồn với các công thức trong bài, phát hiện và sửa những chỗ sai lệch; kiểm chứng kết quả trên các tập dữ liệu chuẩn BEIR và MS MARCO với baseline đối chứng.
- Cải tiến mã nguồn: tối ưu thuật toán truy hồi chính (chấm điểm MaxSim thưa trên inverted index) và so sánh với bản gốc cũng như với ColBERTv2 + PLAID về tốc độ, bộ nhớ và chất lượng trên cùng một máy.
- Đánh giá các tuyên bố của bài báo dựa trên số liệu đo được và rút ra kinh nghiệm về tái lập nghiên cứu.

1.7.2. Phạm vi

- *Lý thuyết:* bài báo arXiv:2605.30120 (đối chiếu các bản v3, v4 và bản chính thức trên PMLR 306), cùng các công trình nền ColBERT, ColBERTv2, PLAID, SPLADE, TopK SAE.

- *Thực hành:* mã nguồn chính thức tại commit `c64bc05` và checkpoint `SSR-tok` trên HuggingFace; các tập NFCorpus, SciFact, FiQA-2018, Quora (BEIR) và MS MARCO passage; baseline đối chứng ColBERTv2 (PLAID) và BM25. Phần huấn luyện lại từ đầu nằm ngoài phạm vi vì vượt quá tài nguyên phần cứng của nhóm (một GPU 8 GB); nhóm chỉ sửa và chạy thử đường huấn luyện.

1.7.3. Phương pháp

- **Tổng hợp và phân tích tài liệu:** dịch và đọc bài báo gốc cùng các công trình nền, tính lại các bảng số liệu trong bài và so sánh các phiên bản của bài.
- **Phân tích mã nguồn:** đọc toàn bộ repository, đối chiếu từng hàm với công thức tương ứng, đọc các issue trên GitHub và trả lời của tác giả.
- **Thực nghiệm có đối chứng:** mọi phép đo chạy trên cùng một máy, dùng cùng pipeline cho SSR và ColBERTv2; pipeline được kiểm tra bằng cách tái lập số đã công bố của ColBERTv2 và BM25 trước khi đo SSR. Mỗi bản sửa được kiểm chứng bằng ít nhất hai cách tính độc lập.
- **Tối ưu và đo lường:** thiết kế lại kernel chấm điểm, viết test so sánh từng bit với bản gốc, đo độ trễ p50/p95 và bộ nhớ.

Ở thời điểm thực hiện báo cáo (10/2026), mã nguồn của bài có hai issue công khai từ hai nhóm độc lập báo không tái lập được Bảng 1 của bài [19]; tác giả đã thừa nhận một số lỗi và bản arXiv v4 đã sửa lại cách so sánh trên MS MARCO. Vì vậy, bên cạnh việc tìm hiểu ý tưởng, kiểm tra khả năng tái lập là nội dung trọng tâm của Chương 5.

1.7.4. Cấu trúc báo cáo

Báo cáo gồm năm chương, đi theo thứ tự trình bày của bài báo: Chương 2 trình bày biểu diễn thưa và late interaction cùng cơ sở lý thuyết; Chương 3 trình bày cách huấn luyện lai sparse projector; Chương 4 trình bày inverted index mức neuron và hai thuật toán SSR, SSR++; Chương 5 tổng hợp kết quả thực nghiệm của bài báo, trình bày quá trình cài đặt, tái lập, sửa lỗi và tối ưu mã nguồn của nhóm, rồi thảo luận và kết luận. Phụ lục A ghi chi tiết từng lỗi đã sửa kèm đoạn mã trước và sau; Phụ lục B là hướng dẫn chạy lại toàn bộ thí nghiệm. Mã nguồn, patch, script đánh giá và kết quả của nhóm được công bố tại https://github.com/duykhanhxx03/ssr-repro-and-optimization.

CHƯƠNG 2. SPARSE REPRESENTATION & LATE INTERACTION

Chương này giải thích ý tưởng cốt lõi của SSR: vì sao cần chuyển từ biểu diễn dense ít chiều sang biểu diễn thưa nhiều chiều, điểm tương đồng được tính như thế nào trên không gian thưa, và vì sao về mặt lý thuyết điểm thưa vẫn xấp xỉ tốt điểm dense với sai số bị chặn.

2.1. Từ biểu diễn dense sang biểu diễn thưa

2.1.1. Hạn chế của biểu diễn dense ít chiều

ColBERT chiếu mỗi token về một vector 128 chiều trong đó hầu như mọi chiều đều khác không. Chính đặc điểm "dense" này khiến inverted index không dùng được: một vector dense "nằm" trên mọi chiều nên mọi token đều liên quan tới mọi chiều, không có chiều nào đủ đặc trưng để làm khóa tra cứu. Muốn tìm ứng viên, hệ thống buộc phải clustering các vector rồi duyệt theo centroid, tức là quay lại nút thắt đã nêu ở mục 1.4.

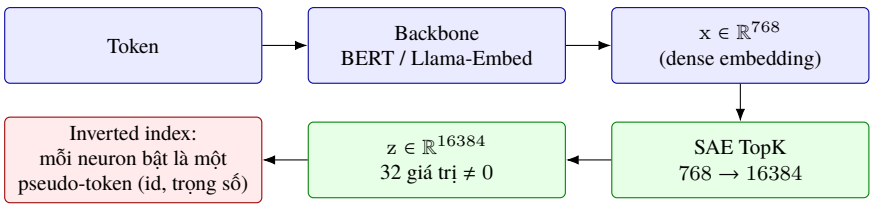
2.1.2. Sparse coding và giả thuyết superposition

Sparse coding [11] biểu diễn một tín hiệu bằng tổ hợp của một số ít phần tử lấy từ một từ điển lớn hơn nhiều so với số chiều tín hiệu (từ điển overcomplete, $h \gg d$). Trong Sparse Autoencoder, mỗi neuron được kỳ vọng đảm nhiệm một "khái niệm" ngữ nghĩa hẹp, và dense embedding của token được xem là chồng chập (superposition) của nhiều khái niệm như vậy. Nhiệm vụ của SAE là tách rời các khái niệm đó ra thành các chiều riêng biệt.

Với truy hồi, điều quan trọng là hai token gần nghĩa sẽ kích hoạt chung một số neuron. Khi đó, một neuron hoạt động đóng vai trò như một pseudo-token, tương tự một từ khóa trong BM25. Vì số neuron bật cho mỗi token rất nhỏ ($K = 32$ trên $h = 16.384$, tức khoảng 0,2%), mỗi neuron chỉ xuất hiện ở một phần nhỏ token, đủ để xây inverted index. Bảng 2.1 đối chiếu hai cách biểu diễn, Hình 2.1 mô tả luồng biến đổi một token.

Bảng 2.1: So sánh biểu diễn dense của ColBERT và biểu diễn thưa của SSR.

Tiêu chí	Dense (ColBERT)	Sparse (SSR)
Số chiều mỗi token	128	16.384
Số phần tử khác không	128 (100%)	32 (≈0,2%)
Ý nghĩa mỗi chiều	Không diễn giải được	Xấp xỉ một khái niệm
Cấu trúc tìm kiếm	Clustering + ANN	Inverted index
Chi phí tích vô hướng một cặp token	128 phép nhân	≤ 32 (chỉ trên phần giao)



Hình 2.1: Luồng biến đổi một token trong SSR: backbone tạo dense embedding, SAE TopK tạo sparse code 16.384 chiều với 32 giá trị khác không, mỗi chiều hoạt động trở thành một pseudo-token và được ghi vào inverted index.

2.2. TopK Sparse Autoencoder (giới thiệu)

Sparse Autoencoder nhận dense embedding $\mathrm{x} \in \mathbb{R}^d$ và tạo sparse code $\mathrm{z} \in \mathbb{R}^h$, sau đó toán tử TopK giữ lại đúng K giá trị lớn nhất và đặt mọi giá trị còn lại về không. Quá trình này được áp dụng độc lập cho từng token của cả truy

vấn lẫn tài liệu:

$$Q' = \{\mathrm{z}_{q_1}, \ldots, \mathrm{z}_{q_N}\} \in \mathbb{R}^{h \times N}, \qquad D' = \{\mathrm{z}_{d_1}, \ldots, \mathrm{z}_{d_M}\} \in \mathbb{R}^{h \times M}. \tag{2.1}$$

Chi tiết kiến trúc và cách huấn luyện SAE được trình bày ở Chương 3; ở chương này chỉ cần hai tính chất: sparse code giữ ít giá trị khác không, và mỗi chiều khác không tương ứng với một neuron mang nghĩa. Vì TopK được áp dụng sau ReLU, mọi giá trị trong z đều *không âm*; tính chất này sẽ được dùng ở mục 2.3 và trong phần tối ưu ở Chương 5.

2.3. Late interaction trên không gian thưa

Điểm liên quan trong SSR là một biến thể thưa của MaxSim. Gọi $\mathcal{A}_K(\mathrm{z})$ là tập *chỉ số* của K neuron có giá trị lớn nhất trong z. Với mỗi token truy vấn, thay vì lấy cực đại trên các token tài liệu theo tích vô hướng đầy đủ, ta chỉ cộng trên tập neuron hoạt động chung:

$$S(Q, D) = \sum_{i=1}^{N} \max_{j=1}^{M} \left(\sum_{u \in \mathcal{A}_K(\mathrm{z}_{q_i}) \cap \mathcal{A}_K(\mathrm{z}_{d_j})} z_{q_i}^{(u)}\, z_{d_j}^{(u)} \right). \tag{2.2}$$

Đây là công thức (4) của bài báo. Đọc công thức từ trong ra ngoài: tổng bên trong chỉ chạy trên phần giao các neuron hoạt động của *một cặp* token truy vấn và token tài liệu; phép max chọn token tài liệu khớp nhất cho mỗi token truy vấn; và tổng ngoài cùng cộng đóng góp của tất cả token truy vấn. Từ đó có hai hệ quả. Nếu phần giao rỗng thì điểm của cặp token bằng không và không cần tính; đây là cơ sở để xây inverted index ở Chương 4. Ngoài ra, chi phí cho mỗi cặp token bị chặn bởi K phép nhân thay vì d phép nhân.

Bảng 2.2: Ví dụ tính tay điểm thưa cho token truy vấn q_1 với hai token tài liệu.

Token	Neuron hoạt động (chỉ số : giá trị)		
q_1 “neural”	12 : 0,9	305 : 0,7	9001 : 0,2
d_1 “network”	12 : 0,8	77 : 0,5	
d_2 “brain”	305 : 0,6	9001 : 0,4	

Với ví dụ ở Bảng 2.2, token q_1 khớp với d_1 chỉ trên neuron 12 nên đóng góp là $0{,}9 \times 0{,}8 = 0{,}72$; khớp với d_2 trên hai neuron 305 và 9001 nên đóng góp là $0{,}7 \times 0{,}6 + 0{,}2 \times 0{,}4 = 0{,}50$. MaxSim lấy giá trị lớn hơn, tức 0,72. Nếu truy vấn có thêm token q_2, tổng $S(Q, D)$ sẽ là tổng các cực đại của từng token truy vấn.

Thứ tự các phép toán. Thứ tự trong (2.2) quan trọng và dễ cài sai: với mỗi cặp (token query i, token tài liệu j) phải *cộng* các tích trên neuron chung trước, rồi mới lấy *max* theo j, cuối cùng cộng theo i. Nếu đảo lại, tức lấy max theo từng neuron trên toàn tài liệu rồi mới gộp, điểm của cặp (token query, tài liệu) trở thành $\max_u q_i^{(u)} \max_j d_j^{(u)}$: phép cộng trên các neuron chung bị mất và các neuron không còn bị ràng buộc phải thuộc cùng một token tài liệu. Ở ví dụ trên, nếu tài liệu gồm cả d_1 và d_2, cách tính sai cho q_1 điểm $\max(0{,}9 \cdot 0{,}8;\ 0{,}7 \cdot 0{,}6;\ 0{,}2 \cdot 0{,}4) = 0{,}72$, trùng với kết quả đúng chỉ vì tình cờ; nếu tài liệu chỉ gồm d_2, nó cho $\max(0{,}42;\ 0{,}08) = 0{,}42$ thay vì $0{,}50$. Mã nguồn công bố của bài báo mắc đúng lỗi này ở chế độ `exact` trên inverted index (mục 5.3.4).

2.4. Hai biến thể SSR-tok và SSR-CLS

SSR huấn luyện hai SAE riêng: $\mathbb{E}_{\text{tok}}$ cho các token thường và $\mathbb{E}_{[\text{CLS}]}$ cho token `[CLS]`. Trên cơ sở đó có hai biến thể:

- **SSR-tok** chỉ dùng công thức (2.2) với sparse code của token thường.
- **SSR-CLS** cộng thêm tín hiệu ngữ nghĩa toàn cục bằng độ tương đồng cosine giữa sparse code của token `[CLS]` ở truy vấn và tài liệu.

Theo bài báo, SSR-CLS luôn cao hơn SSR-tok khoảng 0,5 điểm nDCG@10 nhưng chậm hơn khoảng 2 ms; khi K nhỏ, mô hình phụ thuộc vào `[CLS]` nhiều hơn. Bài báo không nói rõ cách kết hợp điểm `[CLS]` với điểm token (cộng có trọng số hay cách khác), và tác giả chỉ công bố checkpoint SSR-tok. Vì vậy toàn bộ thực nghiệm của nhóm ở Chương 5 dùng SSR-tok.

2.5. Cơ sở lý thuyết: tính chất sai lệch bị chặn

Phần này trả lời câu hỏi: khi thay tích vô hướng dense bằng tích vô hướng thưa thì sai số là bao nhiêu. Nội dung dựa trên Phụ lục A của bài báo.

2.5.1. Thiết lập

Gọi $\mathrm{x}, \mathrm{y} \in \mathbb{R}^d$ là hai dense embedding (đã tâm hóa, tức gộp $\mathrm{b}_{\mathrm{pre}}$ vào embedding), $\mathrm{z}_x, \mathrm{z}_y \in \mathbb{R}^h$ là sparse code TopK tương ứng, và $\hat{\mathrm{x}} = \mathrm{W}_{\mathrm{dec}}\mathrm{z}_x$, $\hat{\mathrm{y}} = \mathrm{W}_{\mathrm{dec}}\mathrm{z}_y$ là các vector tái tạo. Hai điểm số được so sánh là

$$S_{\mathrm{dense}}(Q, D) = \sum_{i=1}^{N} \max_j \mathrm{q}_i^\top \mathrm{d}_j, \tag{2.3}$$

$$S_{\mathrm{SSR}}(Q, D) = \sum_{i=1}^{N} \max_j \mathrm{z}_{q_i}^\top \mathrm{z}_{d_j}, \tag{2.4}$$

trong đó $\mathrm{z}_{q_i}^\top \mathrm{z}_{d_j}$ đúng bằng tổng trong ngoặc của (2.2), vì các thành phần ngoài phần giao đều bằng không.

2.5.2. Bốn giả thiết

Bảng 2.3: Các giả thiết của phân tích sai lệch bị chặn.

#	Giả thiết	Công thức	Ý nghĩa
1	Tái tạo trung thực	$\|\mathrm{x} - \hat{\mathrm{x}}\|_2 \le \epsilon$, $\|\mathrm{y} - \hat{\mathrm{y}}\|_2 \le \epsilon$	SAE tái tạo tốt embedding gốc
2	Chuẩn embedding bị chặn	$\|\mathrm{x}\|_2 \le B$, $\|\mathrm{y}\|_2 \le B$	Embedding không quá lớn
3	Decoder gần trực giao trên tập hoạt động $S = \mathrm{supp}(\mathrm{z}_x) \cup \mathrm{supp}(\mathrm{z}_y)$	$\left\|(\mathrm{W}_{\mathrm{dec}}^\top \mathrm{W}_{\mathrm{dec}} - \mathrm{I})\right\|_{S\to S} \le \delta$	Các cột decoder của neuron đang bật gần như vuông góc, tức mỗi neuron là một hướng độc lập
4	Chuẩn sparse code bị chặn	$\|\mathrm{z}_x\|_2 \le C$, $\|\mathrm{z}_y\|_2 \le C$	Dùng để thu được cận đều theo mọi token

Giả thiết 3 có nghĩa là với mọi vector thưa a, b có support nằm trong S: $|\mathrm{a}^\top(\mathrm{W}_{\mathrm{dec}}^\top \mathrm{W}_{\mathrm{dec}} - \mathrm{I})\mathrm{b}| \le \delta\|\mathrm{a}\|_2\|\mathrm{b}\|_2$.

2.5.3. Định lý A: sai lệch mức token

Định lý A. *Với các Giả thiết 1–3, tích vô hướng của hai dense embedding và tích vô hướng của hai sparse code tương ứng thỏa mãn*

$$\left|\mathrm{x}^\top \mathrm{y} - \mathrm{z}_x^\top \mathrm{z}_y\right| \le 2B\epsilon + \epsilon^2 + \delta\|\mathrm{z}_x\|_2\|\mathrm{z}_y\|_2. \tag{2.5}$$

Nếu Giả thiết 4 cũng đúng thì vế phải không vượt quá $2B\epsilon + \epsilon^2 + \delta C^2$.

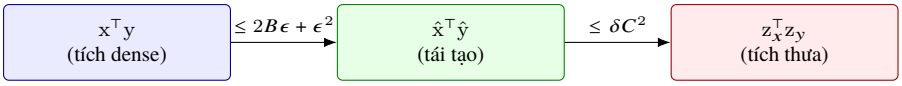
Chứng minh. Chứng minh gồm ba bước (Hình 2.2).

Bước 1: bất đẳng thức tam giác. Tách sai lệch thành phần do tái tạo và phần do cấu trúc decoder:

$$\left|\mathrm{x}^\top \mathrm{y} - \mathrm{z}_x^\top \mathrm{z}_y\right| \le \left|\mathrm{x}^\top \mathrm{y} - \hat{\mathrm{x}}^\top \hat{\mathrm{y}}\right| + \left|\hat{\mathrm{x}}^\top \hat{\mathrm{y}} - \mathrm{z}_x^\top \mathrm{z}_y\right|. \tag{2.6}$$

Bước 2: sai số tái tạo. Đặt $\mathrm{e}_x = \mathrm{x} - \hat{\mathrm{x}}$ và $\mathrm{e}_y = \mathrm{y} - \hat{\mathrm{y}}$. Khai triển $\mathrm{x}^\top \mathrm{y} - \hat{\mathrm{x}}^\top \hat{\mathrm{y}} = \mathrm{x}^\top \mathrm{e}_y + \mathrm{e}_x^\top \mathrm{y} - \mathrm{e}_x^\top \mathrm{e}_y$. Theo bất đẳng thức Cauchy-Schwarz cùng Giả thiết 1 và 2, ba số hạng lần lượt bị chặn bởi $B\epsilon$, ϵB và ϵ^2, nên tổng không vượt quá $2B\epsilon + \epsilon^2$.

Bước 3: sai số hình học của decoder. Vì $\hat{\mathrm{x}} = \mathrm{W}_{\mathrm{dec}} \mathrm{z}_x$ và $\hat{\mathrm{y}} = \mathrm{W}_{\mathrm{dec}} \mathrm{z}_y$ nên $\hat{\mathrm{x}}^\top \hat{\mathrm{y}} = \mathrm{z}_x^\top (\mathrm{W}_{\mathrm{dec}}^\top \mathrm{W}_{\mathrm{dec}}) \mathrm{z}_y$. Trừ đi $\mathrm{z}_x^\top \mathrm{z}_y$ ta được $\mathrm{z}_x^\top (\mathrm{W}_{\mathrm{dec}}^\top \mathrm{W}_{\mathrm{dec}} - \mathrm{I}) \mathrm{z}_y$; theo Giả thiết 3, biểu thức này bị chặn bởi $\delta \|\mathrm{z}_x\|_2 \|\mathrm{z}_y\|_2$, và theo Giả thiết 4 là δC^2. Ghép ba bước lại thu được (2.5). □



Hình 2.2: Phân rã sai số thành hai thành phần: sai số tái tạo của SAE ($\leq 2B\epsilon + \epsilon^2$) và sai số hình học của decoder ($\leq \delta C^2$).

2.5.4. Định lý B: sai lệch mức MaxSim

Định lý B. *Giả sử các Giả thiết 1–4 đúng với mọi cặp token truy vấn và token tài liệu* $(\mathrm{q}_i, \mathrm{d}_j)$ *và đặt* $\eta = 2B\epsilon + \epsilon^2 + \delta C^2$. *Khi đó*

$$\left| S_{\mathrm{dense}}(Q, D) - S_{\mathrm{SSR}}(Q, D) \right| \leq N\eta. \tag{2.7}$$

Chứng minh. Đặt $s_{ij} = \mathrm{q}_i^\top \mathrm{d}_j$ và $\tilde{s}_{ij} = \mathrm{z}_{q_i}^\top \mathrm{z}_{d_j}$. Theo Định lý A, $|s_{ij} - \tilde{s}_{ij}| \leq \eta$ với mọi i, j. Dùng bất đẳng thức sơ cấp $|\max_j a_j - \max_j b_j| \leq \max_j |a_j - b_j|$, với mỗi token truy vấn sai lệch giữa hai cực đại không vượt quá η. Cộng dồn trên N token truy vấn cho ra (2.7). □

2.5.5. Ý nghĩa và nhận xét

Cận sai số cho thấy vài điều. Sai số tăng tuyến tính theo độ dài truy vấn N chứ không bùng nổ, nên hệ thống vẫn ổn định với truy vấn dài. Cận này cũng giải thích vì sao Chương 3 cần reconstruction loss (để ϵ nhỏ) và cần các neuron tách biệt (để δ nhỏ): chất lượng biểu diễn thưa được kiểm soát bởi những đại lượng có thể tối ưu trong huấn luyện.

Nhóm có ba nhận xét về phân tích này. Đây là điều kiện đủ và cận đưa ra có thể khá lỏng so với thực tế. Bài báo không đo ϵ và δ của SAE đã huấn luyện nên chưa kiểm chứng định lượng được cận. Với bài toán xếp hạng, chỉ cần *thứ tự* tài liệu được bảo toàn chứ không cần điểm số gần đúng, nên yêu cầu thực tế yếu hơn cận này. Thực nghiệm của nhóm trên checkpoint công bố cho thấy tích vô hướng giữa các token thưa rất nhỏ (ngưỡng để lọt top-100 chỉ khoảng 0,22–0,32 cho mỗi token truy vấn), nên các cận trên dùng để pruning cũng rất lỏng; mục 5.4.5 phân tích kỹ hơn.

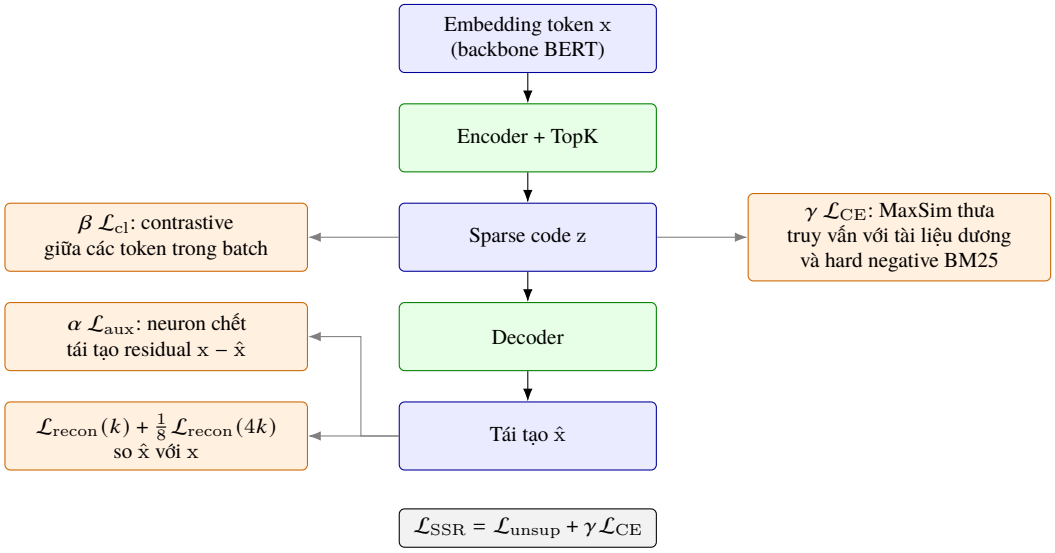
Chương này cho thấy biểu diễn thưa là cơ sở để dùng inverted index, và sai số so với biểu diễn dense bị chặn bởi các đại lượng điều khiển được. Để có sparse code thỏa mãn các điều kiện trên, SAE cần được huấn luyện với hàm mục tiêu phù hợp; đó là nội dung của Chương 3.

CHƯƠNG 3. HYBRID TRAINING OF SPARSE PROJECTORS

Chương này trình bày kiến trúc của Sparse Autoencoder dùng trong SSR và phân tích từng thành phần của hàm mục tiêu huấn luyện: công thức, ý nghĩa, vai trò trong hệ thống và bằng chứng thực nghiệm cho thấy thành phần đó thực sự đóng góp. Cuối chương, nhóm đối chiếu các thiết lập huấn luyện trong bài với mã nguồn công bố và với trả lời của tác giả trên GitHub.

3.1. Yêu cầu đối với sparse projector

Sparse projector phải thỏa mãn đồng thời hai yêu cầu có phần đối lập nhau. Yêu cầu thứ nhất là *tái tạo*: sparse code phải giữ đủ thông tin của embedding gốc để sai số tái tạo ϵ nhỏ, qua đó bảo đảm cận sai lệch ở Định lý A (mục 2.5). Yêu cầu thứ hai là *phân biệt*: sparse code phải đủ đặc trưng để xếp hạng tài liệu liên quan cao hơn tài liệu không liên quan, điều mà chỉ tối ưu tái tạo không bảo đảm, bởi hai embedding hoàn toàn có thể được tái tạo tốt mà không chia sẻ neuron nào. Vì phải thỏa mãn cả hai, quá trình huấn luyện được gọi là *huấn luyện lai*: vừa không giám sát (tái tạo) vừa có giám sát (xếp hạng). Hình 3.1 tổng quan quá trình này.



Hình 3.1: Tổng quan quá trình huấn luyện SAE trong SSR: embedding token từ backbone đi qua encoder và toán tử TopK để tạo sparse code, sau đó được giải mã; hàm mục tiêu gồm bốn nhóm thành phần: tái tạo, chống neuron chết, contrastive giữa các token và contrastive giữa truy vấn và tài liệu.

Về backbone, bài báo có hai thiết lập khác nhau. Với backbone LLM (Llama-Embed-8B), bài nói rõ backbone được *đóng băng* và chỉ SAE được huấn luyện. Với backbone BERT-base dùng cho các kết quả chính, bài không nói rõ; theo trả lời của tác giả trên GitHub (mục 3.7), BERT *không* đóng băng trong thiết lập này, nghĩa là cả backbone lẫn SAE đều được cập nhật.

3.2. TopK Sparse Autoencoder

Kiến trúc SAE [10] gồm encoder, toán tử TopK và decoder:

$$z = \text{TopK}\left(W_{\text{enc}}(x - b_{\text{pre}}) + b_{\text{enc}}\right), \tag{3.1}$$

$$\hat{x} = W_{\text{dec}}\, z + b_{\text{pre}}, \tag{3.2}$$

$$\mathcal{L}_{\text{recon}}(k) = \|x - \hat{x}\|_2^2 \quad \text{(khi z giữ } k \text{ neuron)}. \tag{3.3}$$

Trong đó $\mathrm{W}_{\mathrm{enc}} \in \mathbb{R}^{h \times d}$, $\mathrm{W}_{\mathrm{dec}} \in \mathbb{R}^{d \times h}$; $\mathrm{b}_{\mathrm{pre}}$ đóng vai trò "tâm" của dữ liệu, được trừ trước khi mã hóa và cộng lại sau khi giải mã. Toán tử TopK giữ K giá trị lớn nhất (sau ReLU) và đặt phần còn lại bằng không. So với SAE dùng phạt L_1, cách tiếp cận TopK có ba ưu điểm: kiểm soát chính xác số neuron hoạt động; không gây hiện tượng co giá trị (shrinkage) làm lệch biên độ sparse code; và không cần dò hệ số phạt. Ngoài ra, K cũng là số pseudo-token của mỗi token, nên quyết định trực tiếp kích thước inverted index ở Chương 4. Hai kỹ thuật kèm theo của TopK SAE cũng được dùng: *AuxK* (mục 3.3.3) và *annealing* K từ $2K$ về K trong giai đoạn đầu huấn luyện. SSR dùng hai SAE riêng biệt $\mathbb{E}_{\mathrm{tok}}$ và $\mathbb{E}_{[\mathrm{CLS}]}$ với $d = 768$, $h = 16.384$, $K = 32$, $k_{\mathrm{aux}} = 2.048$ (Bảng 3.1).

Bảng 3.1: Siêu tham số huấn luyện SAE với backbone BERT-base theo Bảng 6 của bài báo.

d	h	K	k_{aux}	Optimizer	lr	Batch	Epoch	Warmup	α	β	γ
768	16.384	32	2.048	AdamW	10^{-3}	64	2	20.000	1/32	0,1	0,05

Với backbone lớn hơn như Llama-Embed-8B, các siêu tham số được điều chỉnh tương ứng: $d = 4.096$, $h = 65.536$, huấn luyện ở định dạng BF16 với $\mathrm{lr} = 2 \cdot 10^{-4}$ và batch toàn cục 32.

3.3. Hàm loss không giám sát

Hàm loss không giám sát gồm bốn thành phần (công thức (7) của bài):

$$\mathcal{L}_{\mathrm{unsup}} = \mathcal{L}_{\mathrm{recon}}(k) + \tfrac{1}{8}\,\mathcal{L}_{\mathrm{recon}}(4k) + \alpha\,\mathcal{L}_{\mathrm{aux}}(k_{\mathrm{aux}}) + \beta\,\mathcal{L}_{\mathrm{cl}}. \tag{3.4}$$

3.3.1. Reconstruction loss $\mathcal{L}_{\mathrm{recon}}(k)$

Đây là thành phần cơ bản nhất, buộc SAE giữ thông tin của embedding gốc khi chỉ cho phép k neuron hoạt động. Vai trò của nó là trực tiếp làm giảm ϵ trong Định lý A. Nếu bỏ thành phần này, sparse code sẽ mất thông tin và cận sai lệch nới rộng.

3.3.2. Multi-TopK: số hạng $\frac{1}{8}\mathcal{L}_{\mathrm{recon}}(4k)$

Thành phần này yêu cầu SAE tái tạo tốt không chỉ ở mức K neuron mà cả khi giữ $4K = 128$ neuron. Nhờ vậy các neuron ngoài top-K vẫn được học, SAE hoạt động tốt ở nhiều mức thưa khác nhau, và có thể thay đổi K lúc suy luận mà không phải huấn luyện lại; đây là nền tảng cho các thí nghiệm K thích ứng ở mục 5.2.5. Hệ số $\frac{1}{8}$ được chọn để thành phần này không lấn át số hạng tái tạo chính.

3.3.3. Auxiliary loss $\mathcal{L}_{\mathrm{aux}}(k_{\mathrm{aux}})$ chống neuron chết

Neuron chết (dead latent) là neuron lâu ngày không lọt vào TopK, do đó không bao giờ được cập nhật và trở thành lãng phí chiều. $\mathcal{L}_{\mathrm{aux}}$ chọn ra $k_{\mathrm{aux}} = 2.048$ neuron "chết lâu nhất" để tái tạo residual $\mathrm{e} = \mathrm{x} - \hat{\mathrm{x}}$, buộc chúng học thông tin hữu ích. Với SSR, hậu quả của neuron chết mang tính hệ thống: một neuron không bao giờ hoạt động tương ứng với một "từ khóa" không được dùng, posting list của nó rỗng; ngược lại nếu một neuron quá phổ biến thì posting list quá dài và làm chậm truy vấn. Vì vậy $\mathcal{L}_{\mathrm{aux}}$ không chỉ tốt cho tái tạo mà còn giúp cân bằng chỉ mục.

3.3.4. Sparse contrastive loss $\mathcal{L}_{\mathrm{cl}}$

$$\mathcal{L}_{\mathrm{cl}} = -\frac{1}{|\mathcal{B}|}\sum_{i=1}^{|\mathcal{B}|} \log \frac{e^{\mathrm{z}_i^{\top}\mathrm{z}_i}}{e^{\mathrm{z}_i^{\top}\mathrm{z}_i} + \sum_{j \neq i} e^{\mathrm{z}_i^{\top}\mathrm{z}_j}}, \tag{3.5}$$

trong đó $\mathcal{B}$ là tập mọi token trong batch. Thành phần này đẩy sparse code của các token khác nhau ra xa nhau, tránh hiện tượng mọi token cùng bật một nhóm neuron (sụp đổ biểu diễn). Nhờ phân bố neuron đều hơn, các posting list trở nên cân bằng về độ dài, giúp ổn định độ trễ truy vấn.

3.4. Supervised contrastive loss $\mathcal{L}_{\mathrm{CE}}$

$$\mathcal{L}_{\mathrm{CE}} = -\log \frac{\exp\big(\mathrm{sim}(Q, D^{+})/\tau\big)}{\sum_{D \in \mathcal{D}_{\mathcal{B}}} \exp\big(\mathrm{sim}(Q, D)/\tau\big)}. \tag{3.6}$$

Mỗi batch $\mathcal{D}_{\mathcal{B}}$ gồm một tài liệu dương D^{+} và các tài liệu âm; tài liệu âm là *hard negative* lấy bằng BM25 trên MS MARCO. Hàm $\mathrm{sim}(\cdot)$ được định nghĩa khác nhau theo SAE: với $\mathbb{E}_{\mathrm{tok}}$ là MaxSim thưa theo công thức (2.2), còn với $\mathbb{E}_{[\mathrm{CLS}]}$ là độ tương đồng cosine. Vai trò của $\mathcal{L}_{\mathrm{CE}}$ là đưa tín hiệu "truy vấn nào khớp tài liệu nào" vào sparse code, điều mà tái tạo không thể bảo đảm. Việc chọn âm từ BM25 có lý do thực tiễn: đó là những tài liệu khớp từ vựng với truy vấn nhưng sai về ngữ nghĩa, buộc mô hình học được khác biệt tinh tế. Công thức (9) trong bài không có nhiệt độ τ; theo tác giả trả lời trên GitHub, giá trị thực dùng là $\tau = 0{,}3$ (mục 3.7).

3.5. Hàm mục tiêu tổng

$$\mathcal{L}_{\mathrm{SSR}} = \mathcal{L}_{\mathrm{unsup}} + \gamma\, \mathcal{L}_{\mathrm{CE}}, \qquad \gamma = 0{,}05. \tag{3.7}$$

Bảng 3.2 tổng hợp các thành phần và vai trò của từng thành phần.

Bảng 3.2: Các thành phần của hàm loss và vai trò của từng thành phần.

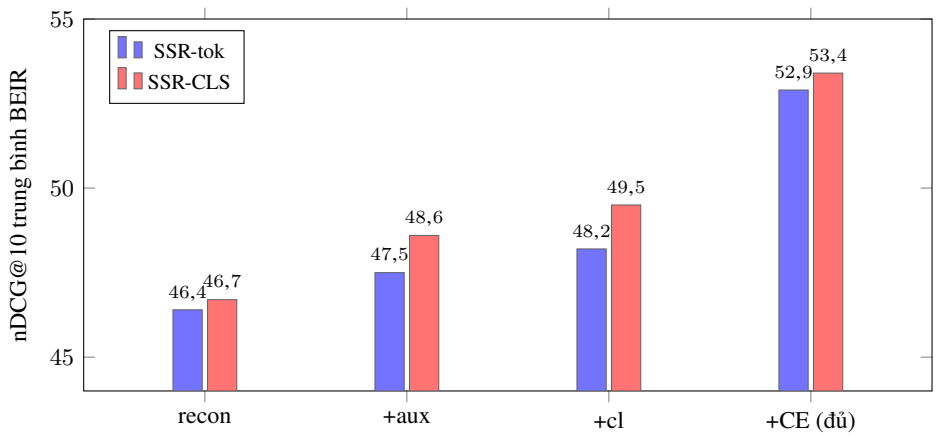
Thành phần	Loại	Trọng số	Tác động chính	Liên hệ lý thuyết / hệ thống
$\mathcal{L}_{\mathrm{recon}}(k)$	Không giám sát	1	Giữ thông tin	ϵ nhỏ (Định lý A)
$\mathcal{L}_{\mathrm{recon}}(4k)$	Không giám sát	1/8	Linh hoạt K	K thích ứng lúc suy luận
$\mathcal{L}_{\mathrm{aux}}$	Không giám sát	$\alpha = 1/32$	Chống neuron chết	Posting list cân bằng
$\mathcal{L}_{\mathrm{cl}}$	Không giám sát	$\beta = 0{,}1$	Đa dạng sparse code	Tránh sụp đổ biểu diễn
$\mathcal{L}_{\mathrm{CE}}$	Có giám sát	$\gamma = 0{,}05$	Khả năng xếp hạng	Mục tiêu truy hồi

3.6. Phân tích thực nghiệm vai trò của từng hàm loss

Để kiểm chứng đóng góp của từng thành phần, ta đọc kết quả ablation của bài báo với $K = 32$, đo nDCG@10 trung bình trên BEIR (Bảng 3.3, Hình 3.2).

Bảng 3.3: Ablation từng thành phần của hàm loss theo bài báo (nDCG@10 trung bình BEIR, $K = 32$).

α	β	γ	SSR-tok	SSR-CLS
0	0	0	46,4	46,7
1/32	0	0	47,5	48,6
1/32	0,1	0	48,2	49,5
1/32	0,1	0,05	**52,9**	**53,4**

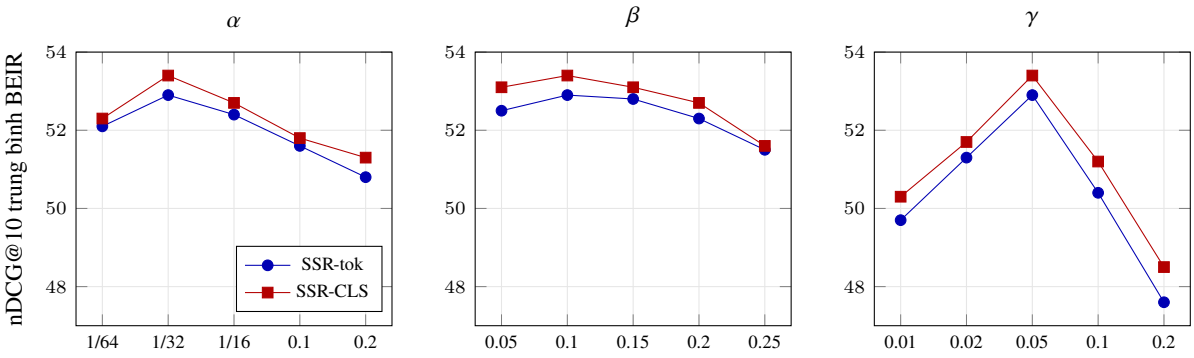


Hình 3.2: Đóng góp tích lũy của từng thành phần hàm loss cho SSR-tok và SSR-CLS (vẽ lại từ số liệu Bảng 3.3).

Kết quả cho thấy đường cơ sở tái tạo thuần túy chỉ đạt 46,4 (SSR-tok) và 46,7 (SSR-CLS). Thêm $\mathcal{L}_{\text{aux}}$ giúp tăng lần lượt 1,1 và 1,9 điểm; thêm $\mathcal{L}_{\text{cl}}$ tăng tiếp 0,7 và 0,9 điểm; và khi thêm $\mathcal{L}_{\text{CE}}$, điểm nhảy lên 52,9 và 53,4. Như vậy $\mathcal{L}_{\text{CE}}$ là thành phần đóng góp lớn nhất, cộng thêm khoảng 4,7 điểm cho SSR-tok và 3,9 điểm cho SSR-CLS.

Bảng 3.4: Độ nhạy của nDCG@10 trung bình BEIR theo ba trọng số α, β, γ (theo bài báo).

α	1/64	1/32	1/16	0,1	0,2
SSR-tok	52,1	**52,9**	52,4	51,6	50,8
SSR-CLS	52,3	**53,4**	52,7	51,8	51,3
β	0,05	0,1	0,15	0,2	0,25
SSR-tok	52,5	**52,9**	52,8	52,3	51,5
SSR-CLS	53,1	**53,4**	53,1	52,7	51,6
γ	0,01	0,02	0,05	0,1	0,2
SSR-tok	49,7	51,3	**52,9**	50,4	47,6
SSR-CLS	50,3	51,7	**53,4**	51,2	48,5



Hình 3.3: Độ nhạy của chất lượng theo ba trọng số α, β, γ (vẽ lại từ Bảng 3.4).

Phân tích Bảng 3.4 và Hình 3.3 cho thấy γ là trọng số nhạy nhất: khi tăng lên 0,2, chất lượng giảm khoảng 5 điểm. Nguyên nhân là $\mathcal{L}_{\text{CE}}$ lấn át tái tạo, khiến ϵ tăng và mô hình mất thông tin; diễn biến này khớp với Định lý A. Hai trọng số α và β ít nhạy hơn, dao động không quá hai điểm. Bài báo chưa làm ablation riêng cho thành phần $\mathcal{L}_{\text{recon}}(4k)$, trong khi đây lại là thành phần bị thiếu trong mã nguồn công bố (mục 3.7).

3.7. Đối chiếu thiết lập huấn luyện với mã nguồn công bố

Khi đối chiếu phần huấn luyện của repository với Bảng 6 và các công thức (7)–(10), nhóm thấy nhiều điểm lệch. Hai nhóm độc lập đã mở issue trên GitHub [19] sau khi huấn luyện lại theo repo chỉ đạt NFCorpus nDCG@10 0,017–0,024 (so với 0,388 trong bài). Trong trả lời ngày 31/07/2026 và 04/08/2026, tác giả xác nhận mã công bố nạp SAE khởi tạo ngẫu nhiên, chỉ dùng một hard negative mỗi truy vấn thay vì 32, và thiếu số hạng $\frac{1}{8}\mathcal{L}_{\text{recon}}(4k)$; tác giả cũng cho biết một số giá trị thực dùng khác Bảng 6. Bảng 3.5 đối chiếu ba nguồn.

Nhóm đã sửa các điểm trên trong bản patch (chi tiết ở Phụ lục A): thêm thành phần `reconstruction_4k`, tham số warmup và số hard negative, một `QueryGroupedBatchSampler` gom các triplet của cùng truy vấn vào một batch như tác giả mô tả, và hai script preset `train_table6.sh` (đúng Bảng 6) và `train_issue1_setup.sh` (theo trả lời của tác giả). Đường huấn luyện đã chạy thử trọn vẹn trên một tập MS MARCO nhỏ (batch 16, 8 hard negative mỗi truy vấn; log cho thấy số hạng $4k$ và warmup hoạt động, checkpoint nạp lại được). Huấn luyện đầy đủ cần khoảng $32 \times 500\text{k}/64 \approx 250\text{k}$ bước mỗi epoch với batch 64, vượt quá GPU 8 GB của nhóm nên không thực hiện.

Điều này ảnh hưởng trực tiếp đến phần thực nghiệm: checkpoint `SSR-tok` mà tác giả công bố *không thể* được tạo ra từ mã nguồn công bố ở trạng thái hiện tại, và một số thiết lập thật (lr, τ, số negative) chỉ được biết qua issue. Mọi đánh giá ở Chương 5 vì vậy dùng checkpoint công bố, và các kết luận về chất lượng chỉ áp dụng cho checkpoint đó.

Bảng 3.5: Thiết lập huấn luyện: bài báo, mặc định của mã nguồn công bố, và trả lời của tác giả trong issue. Cột cuối là cách xử lý của nhóm.

Thiết lập	**Bài báo (Bảng 6)**	**Mã nguồn công bố**	**Tác giả nói trong issue**	**Nhóm xử lý**
Số hạng $\frac{1}{8}\mathcal{L}_{\text{recon}}(4k)$	Có	Tính trong forward nhưng không đưa vào loss	Có dùng, chỉ sót khi công bố	Thêm lại (hệ số 0,125)
γ ($\mathcal{L}_{\text{CE}}$)	0,05	0 (tắt)	0,05	Preset đặt 0,05
Nhiệt độ τ	Không ghi	Không có	0,3	Thêm tham số
Hard negative mỗi query	Không ghi	1	32, gom cùng batch (2 query × 32 triplet = 64)	Thêm `--negatives-per-query` và batch sampler gom theo query
Learning rate	10^{-3}	$2 \cdot 10^{-5}$	10^{-3} "không ổn định", khoảng 10^{-6} tốt hơn	Hai preset: theo Bảng 6 và theo issue
Batch / epoch / warmup	64 / 2 / 20.000	256 / 1 / 0	64 / 2 / 20.000 (≈4% của 2 epoch)	Thêm `--warmup-steps`; preset
k_{aux}	2.048	1.024	–	Preset đặt 2.048
BERT đóng băng?	Không ghi	Không đóng băng	Không đóng băng	Giữ nguyên
Nạp checkpoint	–	Tạo SAE ngẫu nhiên 3.072 latent	Thừa nhận lỗi	Viết lại hàm nạp

CHƯƠNG 4. INDEXING & RETRIEVAL: SSR VÀ SSR++

Chương này giải thích cấu trúc inverted index mức neuron, hai thuật toán truy hồi SSR và SSR++, phân tích độ phức tạp, và cuối cùng chỉ ra những điểm cần chú ý khi cài đặt mà nhóm rút ra từ việc đối chiếu mã nguồn với bài báo.

4.1. Nhắc lại inverted index trong lexical retrieval

Trong lexical retrieval, mỗi từ có một danh sách gồm các cặp (docID, trọng số) cho biết tài liệu nào chứa từ đó và với trọng số bao nhiêu. Khi truy vấn, hệ thống chỉ cần duyệt danh sách của những từ xuất hiện trong truy vấn. Để tăng tốc, kỹ thuật Block-Max WAND lưu thêm cận trên của từng block và dùng nó để bỏ qua những tài liệu không thể lọt vào top-k. SSR++ kế thừa hai ý tưởng này: duyệt theo posting list và bỏ block theo cận trên.

4.2. Inverted index mức neuron

SSR xây một posting list cho mỗi neuron. Gọi z_t là sparse code của token t trong tài liệu D; với mỗi neuron $u \in \{1, \dots, h\}$, giá trị đóng góp của tài liệu D vào neuron u là cực đại trên mọi token:

$$\mu_{D,u} = \max_{t \in D} z_t^{(u)}. \tag{4.1}$$

Posting list của neuron u tập hợp mọi tài liệu có đóng góp dương:

$$\mathcal{I}_u = \{(D, \mu_{D,u}) \mid \mu_{D,u} > 0\}. \tag{4.2}$$

Mỗi danh sách được chia thành các block B có kích thước cố định (64 phần tử theo Phụ lục D.1 của bài), và mỗi block lưu cận trên

$$U_B = \max_{D \in B} \mu_{D,u}. \tag{4.3}$$

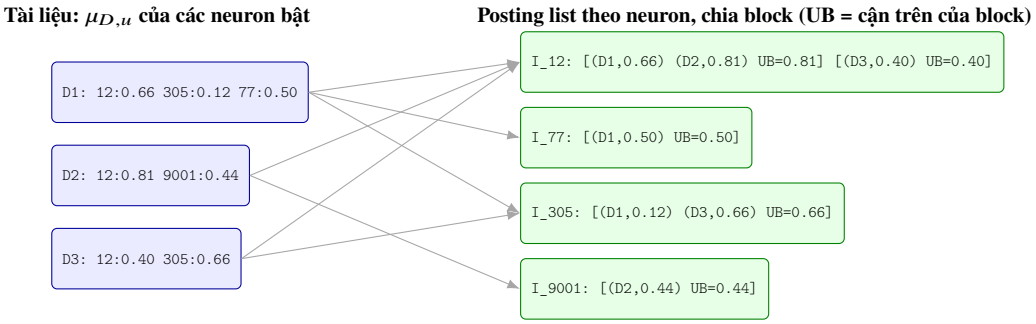
Việc lấy giá trị lớn nhất theo tài liệu đi theo cách nghĩ của MaxSim: ta quan tâm token khớp nhất của tài liệu chứ không phải mọi token. Tổng cộng có $h = 16.384$ danh sách. Thuật toán 4.1 mô tả quy trình lập chỉ mục, Hình 4.1 minh họa cấu trúc.

Thuật toán 4.1 Xây inverted index mức neuron

Đầu vào: corpus $\mathcal{D}$, backbone, SAE $\mathbb{E}_{\text{tok}}$, kích thước block $b = 64$
Đầu ra: các danh sách $\{\mathcal{I}_u\}$ và cận trên $\{U_B\}$

for mỗi tài liệu $D \in \mathcal{D}$ **do**
  $X \leftarrow \text{backbone}(D)$ ▷ embedding mọi token
  $Z \leftarrow \text{TopK}(\mathbb{E}_{\text{tok}}(X))$ ▷ sparse code, $K = 32$ neuron mỗi token
  for mỗi neuron $u \in \bigcup_{t \in D} \mathcal{A}_K(z_t)$ **do**
    $\mu \leftarrow \max_{t \in D} z_t^{(u)}$; thêm (D, μ) vào $\mathcal{I}_u$
  end for
end for
for $u = 1, \dots, h$ **do**
  chia $\mathcal{I}_u$ thành các block b phần tử; lưu $U_B = \max \mu$ trong mỗi block
end for

Forward index. Công thức (4.1) chỉ là bản tóm tắt theo tài liệu. Để tính đúng (2.2), hệ thống vẫn cần giá trị theo *từng token*: posting phải mang bộ ba (tài liệu, token, giá trị), hoặc phải lưu thêm sparse code đầy đủ của từng token



Mỗi block trong hình chứa 2 phần tử để minh họa; cấu hình của bài dùng block 64 phần tử.

Hình 4.1: Cấu trúc inverted index mức neuron: mỗi tài liệu đóng góp giá trị $\mu_{D,u}$ vào posting list của từng neuron mà nó kích hoạt; mỗi posting list chia thành block và mỗi block lưu cận trên U_B.

trong một forward index. Nếu chỉ lưu giá trị cực đại theo tài liệu thì bước tinh của SSR++ chỉ còn là xấp xỉ. Mã nguồn công bố có lưu posting mức token, nhưng phần chấm điểm lại dùng chúng như posting mức tài liệu (mục 4.7).

Bảng 4.1: So sánh chi phí lập chỉ mục trên MS MARCO theo bài báo.

Tiêu chí	ColBERTv2/PLAID	XTR/WARP	SSR
Bước tốn kém nhất	K-means + residual	K-means + residual	Mã hóa + ghi posting
Thời gian (CPU)	122,9 h	103,7 h	7,3 h (tok) / 7,8 h (CLS)
Bộ nhớ đỉnh	274,2 GB	186,7 GB	34,6 GB (tok) / 55,1 GB (CLS)
Kích thước chỉ mục	22,1 GB	55,9 GB	18,5 GB
Thêm tài liệu mới	Xây lại	Xây lại	Chỉ nối thêm (append-only)

4.3. Truy hồi một giai đoạn (SSR)

Với mỗi token truy vấn q_i, hệ thống lấy tập $K = 32$ neuron hoạt động $\mathcal{A}_K(q_i)$, duyệt hợp của $N \times K$ posting list, tích lũy điểm vào từng tài liệu, rồi tính MaxSim thưa chính xác theo (2.2). Ưu điểm là chính xác và chỉ cần một giai đoạn duy nhất, không có nhiều tầng phức tạp. Nhược điểm là $N \times K$ danh sách vẫn khá nhiều: trên MS MARCO, theo bài báo SSR chạm tới khoảng 54.278 ứng viên và mất 38,6 ms mỗi truy vấn.

4.4. Truy hồi tăng tốc SSR++ (từ thô đến tinh)

SSR++ giảm chi phí bằng cách chia truy hồi thành hai bước (Thuật toán 4.2, Hình 4.2).

4.4.1. Bước 1: chấm điểm thô

$$\hat{S}_{\text{coarse}}(Q, D) = \sum_{i=1}^{N} \sum_{u \in \mathcal{A}_{K_{\text{coarse}}}(q_i)} q_i^{(u)}\, \mu_{D,u}. \tag{4.4}$$

Bước thô chỉ dùng $K_{\text{coarse}} = 4$ neuron lớn nhất của mỗi token truy vấn và posting μ. Khi duyệt danh sách theo block, nếu cận trên U_B của block không đủ để tài liệu vượt ngưỡng top-k hiện tại thì bỏ qua toàn bộ block. Kết quả là tập ứng viên C_1 gồm 2.000 tài liệu có điểm thô cao nhất (Phụ lục D.1 của bài).

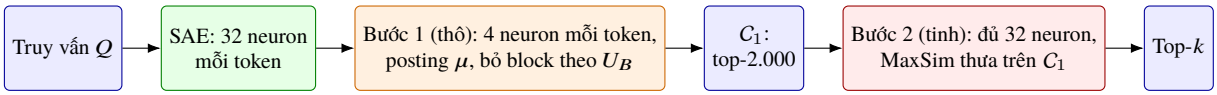
4.4.2. Bước 2: tinh chỉnh chính xác

Với các tài liệu trong C_1, hệ thống dùng đủ $K = 32$ neuron và tính MaxSim thưa theo (2.2) để có thứ hạng cuối cùng. Nhờ vậy, thứ hạng dựa trên đầy đủ K đặc trưng, nhưng phép tính tốn kém chỉ áp dụng cho một phần nhỏ kho.

Thuật toán 4.2 SSR++: truy hồi từ thô đến tinh

Đầu vào: truy vấn Q, chỉ mục $\{\mathcal{I}_u, U_B\}$, forward index, $K_{\text{coarse}} = 4$, $|C_1| = 2000$, $K = 32$
Đầu ra: top-k tài liệu

$Z_q \leftarrow \text{TopK}(\mathbb{E}_{\text{tok}}(\text{backbone}(Q)))$
score $\leftarrow \{\}$ ▷ bước thô
for $i = 1, \ldots, N$ **do**
  for mỗi u trong K_{coarse} neuron lớn nhất của z_{q_i} **do**
    for mỗi block B trong $\mathcal{I}_u$ **do**
      if $z_{q_i}^{(u)} \cdot U_B$ không thể đưa tài liệu vào top-$|C_1|$ **then**
        bỏ qua B
      end if
      for mỗi $(D, \mu) \in B$ **do**
        score$[D]$ += $z_{q_i}^{(u)} \cdot \mu$
      end for
    end for
  end for
end for
$C_1 \leftarrow$ 2.000 tài liệu có score cao nhất
for mỗi $D \in C_1$ **do**
  $S[D] \leftarrow \text{SparseMaxSim}(Z_q, Z_D)$ ▷ bước tinh, công thức (2.2)
end for
return top-k theo S



Hình 4.2: Sơ đồ SSR++: bước chấm điểm thô với 4 neuron mỗi token và pruning theo cận trên của block, rồi bước tinh chỉnh với đủ 32 neuron trên 2.000 ứng viên.

4.4.3. Nhận xét về tính xấp xỉ của bước thô

Bước thô *không phải* là MaxSim. Nó cộng các đóng góp $\mu_{D,u}$ trên các neuron mà không quan tâm chúng có đến từ cùng một token tài liệu hay không, nên có thể đánh giá cao một tài liệu gồm nhiều token khớp rời rạc. Mặt khác, vì chỉ dùng $K_{\text{coarse}} = 4$ trong $K = 32$ neuron của mỗi token truy vấn, nó bỏ qua đóng góp của 28 neuron còn lại, nên cũng *không phải cận trên* của (2.2) như bài báo gọi ("điểm cận trên xấp xỉ"). Vì vậy bước tinh là bắt buộc, và độ chính xác cuối cùng phụ thuộc vào việc tài liệu đúng có lọt vào C_1 hay không, tức recall của C_1. Bài báo cho rằng mức mất là không đáng kể (Bảng 4.3); thực nghiệm của nhóm ở mục 5.3.8 cho thấy điều này chỉ đúng khi 2.000 ứng viên phủ gần hết corpus, còn trên corpus 57k tài liệu SSR++ mất 4,6 điểm nDCG@10.

4.5. Phân tích độ phức tạp

$$\text{SSR:} \quad O(N \cdot K \cdot \bar{L}), \tag{4.5}$$

$$\text{SSR++ (bước thô):} \quad O(N \cdot K_{\text{coarse}} \cdot L_{\text{skip}}), \qquad K_{\text{coarse}} \ll K,\ L_{\text{skip}} \ll \bar{L}, \tag{4.6}$$

trong đó $\bar{L}$ là độ dài trung bình của posting list và L_{skip} là số phần tử thực sự phải duyệt sau khi bỏ block. Bảng 4.2 so sánh với PLAID.

Độ dài trung bình $\bar{L}$ phụ thuộc vào phân bố kích hoạt neuron. Nếu có neuron quá phổ biến (tương tự các từ dừng trong BM25), danh sách của nó rất dài và làm chậm truy vấn; ngược lại neuron quá hiếm làm tăng số chiều lãng phí. Đây là lý do các hàm $\mathcal{L}_{\text{aux}}$ và $\mathcal{L}_{\text{cl}}$ ở Chương 3 có vai trò hệ thống chứ không chỉ là kỹ thuật huấn luyện.

Bảng 4.2: So sánh độ phức tạp giữa PLAID, SSR và SSR++.

Phương pháp	Sinh ứng viên	Chấm điểm cuối
PLAID	Tìm centroid gần nhất cho N token, gom tài liệu	MaxSim dense $O(N \cdot M \cdot d)$ trên ứng viên sau giải nén
SSR	Duyệt $N \cdot K$ danh sách, độ dài trung bình $\bar{L}$	MaxSim thưa $O(N \cdot M \cdot K)$, thực tế nhỏ hơn nhiều vì phần giao thưa
SSR++	Duyệt $N \cdot K_{\text{coarse}}$ danh sách, bỏ block theo cận trên	MaxSim thưa trên 2.000 ứng viên

Bảng 4.3: So sánh SSR và SSR++ trên MS MARCO passage theo bài báo (Bảng 5).

Phương pháp	Số ứng viên	Độ trễ (ms)	MRR@10
SSR	54.278	38,6	45,3
SSR++	3.196	17,5	45,2

4.6. Hiệu quả của SSR++ theo bài báo

Theo Bảng 4.3, SSR++ giảm số ứng viên khoảng 17 lần và giảm độ trễ khoảng 2,2 lần, trong khi MRR@10 chỉ giảm 0,1 điểm. Kết hợp với tính chất append-only, SSR++ được bài báo định vị cho các kho dữ liệu động, nơi tài liệu được thêm liên tục. Số ứng viên 3.196 trong bảng lớn hơn $|C_1| = 2.000$ của Phụ lục D.1, nên cấu hình dùng cho Bảng 5 có thể khác cấu hình được mô tả.

4.7. Những điểm cần chú ý khi cài đặt

Đối chiếu mã nguồn công bố với các công thức trong chương này, nhóm rút ra bốn điểm mà người cài đặt dễ mắc lỗi; mã nguồn của tác giả mắc cả bốn (chi tiết ở mục 5.3.4 và Phụ lục A):

1. **Thứ tự cộng rồi max trong** (2.2)**.** Bộ cộng dồn trên inverted index phải gom theo hàng (tài liệu, token tài liệu) rồi mới lấy max theo token; nếu lấy max theo từng neuron trên toàn tài liệu rồi gộp, kết quả là $\sum_i \max_u q_i^{(u)} \mu_{D,u}$ chứ không phải MaxSim (mục 2.3).

2. **Hệ chỉ số neuron của truy vấn và chỉ mục.** Khi chỉ mục nén các neuron thực sự xuất hiện trong corpus thành chỉ số liên tiếp, truy vấn phải được ánh xạ sang cùng hệ chỉ số; quên bước này làm phần giao neuron gần như rỗng.

3. **Bước thô của SSR++.** Phải cài đúng (4.4) trên posting μ với $K_{\text{coarse}} = 4$, block 64, 2.000 ứng viên toàn cục; tham số khác sẽ cho một thuật toán khác với bài.

4. **Bộ nhớ khi dựng chỉ mục.** Với hàng trăm triệu posting (FiQA: 258 triệu), không thể tạo một object Python cho mỗi posting; cần đếm và điền bằng mảng numpy hoặc kernel biên dịch.

Đến đây, các thành phần của SSR từ biểu diễn, chỉ mục đến truy hồi đã được trình bày đủ. Chương 5 tổng hợp kết quả thực nghiệm của bài báo, trình bày quá trình cài đặt và tái lập của nhóm, các cải tiến mã nguồn, rồi thảo luận và kết luận.

CHƯƠNG 5. EXPERIMENTS, CODE IMPROVEMENT & CONCLUSION

Chương này gồm bốn phần: tổng hợp và phân tích kết quả thực nghiệm của bài báo (mục 5.1 và 5.2); quá trình cài đặt, kiểm chứng và tái lập của nhóm, bao gồm các lỗi đã phát hiện và sửa trong mã nguồn (mục 5.3); phần cải tiến mã nguồn và tối ưu thuật toán truy hồi (mục 5.4); cuối cùng là thảo luận, kết luận và hướng phát triển (mục 5.5 và 5.6). Trong suốt chương, nhóm phân biệt rõ hai loại số liệu: số liệu của tác giả được ghi kèm "theo bài báo", còn số liệu do nhóm tự chạy được ghi kèm "nhóm đo".

5.1. Thiết lập thực nghiệm của bài báo

5.1.1. Bộ dữ liệu

Bảng 5.1: Các bộ dữ liệu dùng trong đánh giá của bài báo.

Bộ	Viết tắt	Lĩnh vực / nhiệm vụ	Vai trò
MS MARCO passage	MS	Truy vấn Bing, 8,84 triệu đoạn	Huấn luyện + in-domain
ArguAna	AR	Phản biện lập luận	BEIR (zero-shot)
Climate-FEVER	CF	Kiểm chứng khí hậu	BEIR
TREC-COVID	CV	Y sinh COVID	BEIR
DBpedia	DB	Tìm thực thể	BEIR
FEVER	FE	Kiểm chứng sự kiện	BEIR
FiQA-2018	FQ	Tài chính	BEIR
HotpotQA	HQ	Hỏi đáp nhiều bước	BEIR
NFCorpus	NF	Y học dinh dưỡng	BEIR
NQ	NQ	Hỏi đáp Google	BEIR
Quora	QU	Câu hỏi trùng lặp	BEIR
SCIDOCS	SD	Trích dẫn khoa học	BEIR
SciFact	SF	Kiểm chứng khoa học	BEIR
Touché-2020	TO	Tranh luận	BEIR
LoTTE	–	Chủ đề đuôi dài, 5 miền	Độ bền long-tail
MS MARCO document	–	Văn bản dài (≈1.131 token)	Văn bản dài
LIMIT	–	Kiểm tra giới hạn biểu diễn	Stress test

BEIR [12] gồm 18 tập dữ liệu truy hồi zero-shot; bài dùng 13 tập. MS MARCO passage [13] có tập dev-small 6.980 truy vấn (nhãn nhị phân) và TREC DL 2019 với 43 truy vấn có nhãn 4 mức.

5.1.2. Độ đo

$$\mathrm{MRR@10} = \frac{1}{|Q|}\sum_{q\in Q}\frac{1}{\mathrm{rank}_q} \quad (\text{bằng 0 nếu } \mathrm{rank}_q > 10), \tag{5.1}$$

$$\mathrm{DCG@10} = \sum_{i=1}^{10}\frac{2^{\mathrm{rel}_i}-1}{\log_2(i+1)}, \qquad \mathrm{nDCG@10} = \frac{\mathrm{DCG@10}}{\mathrm{IDCG@10}}, \tag{5.2}$$

trong đó rel_i là mức liên quan của kết quả thứ i và IDCG là DCG của xếp hạng lý tưởng [16]. BEIR dùng mức liên quan có trọng số (NFCorpus có 3 mức, TREC DL có 4 mức), nên nDCG phải tính với $2^{\mathrm{rel}}-1$ chứ không phải nhãn nhị phân. Ngoài ra bài còn dùng Recall@k và Success@5 (cho LoTTE: tỉ lệ truy vấn có ít nhất một tài liệu đúng trong top-5). Độ trễ được đo bằng mili-giây trên mỗi truy vấn trên GPU A100 40 GB, sau 1.000 truy vấn khởi động; các số liệu CPU đo trên Xeon Platinum 8275CL 96 lõi.

5.1.3. Phương pháp so sánh và cấu hình

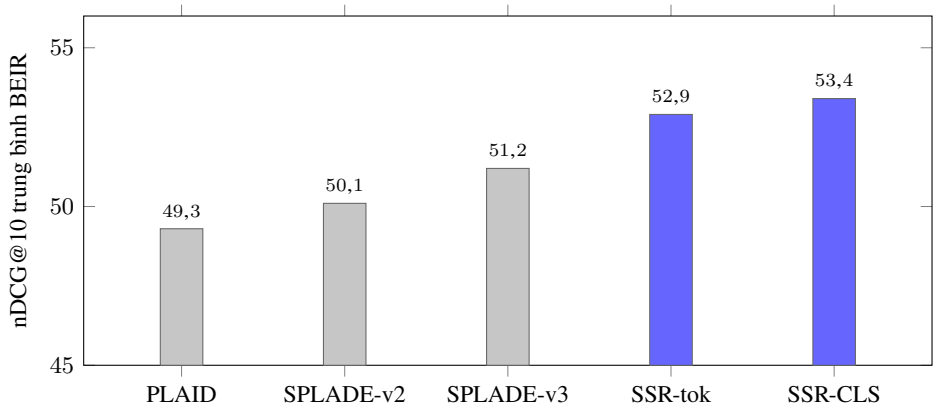
Các hệ dense MVR được so sánh gồm ColBERT, ColBERTv2, PLAID, COIL, AligneR, CITADEL và XTR; các hệ thưa học được gồm SPLADE-v2 và SPLADE-v3. Tất cả được huấn luyện trên MS MARCO với hard negative từ BM25. Cấu hình SSR dùng backbone BERT-base-uncased, $K = 32$, $h = 16.384$, độ dài truy vấn tối đa 32; SSR++ dùng kích thước block 64, $K_{\text{coarse}} = 4$ và giữ top-2.000 ứng viên. Trừ khi ghi rõ, độ trễ của SSR-tok và SSR-CLS trong các bảng là độ trễ của SSR++.

5.2. Kết quả của bài báo

5.2.1. Hiệu năng trong điều kiện có kiểm soát

Bảng 5.2: Bảng 1 của bài báo (bản arXiv v3): nDCG@10 trên 13 tập BEIR, cột MS là MS MARCO (xem mục 5.2.6 về độ đo của cột này), độ trễ ms/truy vấn trên MS MARCO. In đậm: tốt nhất; gạch chân: thứ hai.

Phương pháp	Độ trễ	MS	AR	CF	CV	DB	FE	FQ	HQ	NF	NQ	QU	SD	SF	TO	TB.
Dense retrieval với late interaction																
ColBERT	57,3	36,0	23,6	18,1	67,9	39,2	76,9	31,7	59,2	30,7	51,8	85,7	14,6	66,8	20,5	41,5
COIL	12,6	35,3	29,5	21,7	66,8	39,9	**83,8**	31,3	**71,3**	33,2	52,1	83,8	15,5	70,7	28,1	47,4
ColBERTv2	56,9	39,8	46,3	17,3	73,5	44,9	78,7	36,1	66,6	33,5	56,3	85,3	15,2	69,1	26,1	49,2
PLAID	37,1	39,7	46,1	17,4	73,7	44,7	78,8	36,3	66,7	33,8	56,4	85,0	15,4	69,2	26,3	49,3
AligneR	51,8	38,8	28,9	18,2	68,8	41,6	72,4	33,5	61,5	34,0	52,4	82,3	14,3	70,3	34,8	46,6
CITADEL	15,7	39,9	**51,1**	18,3	71,5	42,2	76,6	33,0	66,3	33,7	54,0	85,3	15,9	69,0	34,2	49,4
XTR	33,4	45,0	40,7	20,8	73,6	40,8	73,7	34,8	64,7	34,0	52,8	85,9	14,6	71,1	31,3	48,8
Learned sparse retrieval																
SPLADE-v2	16,3	43,3	47,6	**23,5**	71,5	43,4	78,7	33,8	68,4	33,5	52,1	83,8	15,9	69,4	**36,3**	50,1
SPLADE-v3	16,6	43,9	50,8	23,3	74,8	45,2	79,8	37,5	69,2	35,8	58,6	81,4	15,8	71,0	29,6	51,2
Sparse retrieval với late interaction																
SSR-tok	17,5	45,2	46,1	22,8	76,4	48,2	81,9	39,4	69,5	38,8	60,7	88,3	17,5	72,9	33,1	52,9
SSR-CLS	19,5	**45,5**	46,6	**23,5**	**76,8**	**48,4**	82,8	**40,0**	69,9	**39,1**	**61,2**	**88,7**	**17,9**	**73,3**	33,7	**53,4**



Hình 5.1: nDCG@10 trung bình trên BEIR của các phương pháp tiêu biểu theo bài báo (vẽ lại từ Bảng 5.2).

Theo Bảng 5.2, SSR-CLS đạt trung bình cao nhất với 53,4, hơn SPLADE-v3 khoảng 2,2 điểm và hơn PLAID khoảng 4,1 điểm; SSR-tok với 17,5 ms nhanh gần gấp đôi ColBERTv2 và PLAID; và SSR đứng đầu trên 9 trong 13 tập BEIR. Về điểm yếu, SSR thấp hơn SPLADE-v3 trên ArguAna (46,6 so với 50,8) và thấp hơn SPLADE-v2 trên Touché (33,7 so với 36,3), còn trên Climate-FEVER thì ngang bằng. Nhóm cho rằng nguyên nhân nằm ở đặc thù miền: các truy vấn tranh luận và quan điểm thường dài, cần ngữ nghĩa toàn cục hoặc lập luận ngược chiều, trong khi khớp từ vựng chính xác lại quan trọng hơn, đúng thế mạnh của họ phương pháp thưa từ vựng như SPLADE.

Bảng 5.3: Kết quả theo bài báo khi dùng Llama-embed-nemotron-8b làm backbone đóng băng (trung bình nDCG@10 trên 14 tập).

Mô hình	Trung bình
Llama-embed-nemotron-8b (gốc)	65,8
Qwen3-Embedding-8B	66,0
e5-mistral-7b	58,8
Llama-8B + lớp chiếu tuyến tính	66,1
Llama-8B + SSR-tok	66,4
Llama-8B + SSR-CLS	**67,1**

5.2.2. Khả năng mở rộng với backbone LLM

Với backbone Llama-8B đóng băng, lớp chiếu tuyến tính chỉ cải thiện khoảng 0,3 điểm trong khi sparse coding cải thiện khoảng 1,3 điểm; bài báo kết luận lợi ích đến từ bản chất sparse coding chứ không phải từ việc thêm tham số. Cải thiện rõ nhất xuất hiện trên DBpedia và FEVER, nơi việc khớp thực thể là then chốt.

5.2.3. Các đánh giá bổ sung: LoTTE, văn bản dài và LIMIT

Bảng 5.4: Success@5 trên LoTTE theo bài báo cho hai loại truy vấn Search và Forum.

Loại	Mô hình	Writing	Recreation	Science	Technology	Lifestyle	Pooled	TB.
Search	ColBERTv2	79,2	70,8	55,5	65,9	84,3	71,8	71,3
	XTR	78,6	69,5	56,1	64,6	83,5	69,3	70,3
	SSR	**81,8**	**73,9**	**59,3**	**67,1**	**87,6**	**73,3**	**73,8**
Forum	ColBERTv2	78,7	71,4	45,8	53,9	77,3	63,5	65,1
	XTR	76,4	70,9	44,2	53,4	75,8	61,7	63,6
	SSR	**81,8**	**74,3**	**47,3**	**55,9**	**79,7**	**67,4**	**67,3**

Trên LoTTE, SSR vượt ColBERTv2 2,5 và 2,2 điểm với truy vấn Search và Forum, khoảng cách tăng lên 3,9 điểm trong thiết lập Pooled. Với văn bản dài trên MS MARCO Document, SSR-CLS đạt MRR@10 48,8 với độ trễ 29,3 ms và SSR-tok đạt 48,3 với 27,5 ms, trong khi ColBERTv2 đạt 41,5 với 79,3 ms; XTR đạt 48,1 còn COIL đạt 38,8 với 18,8 ms.

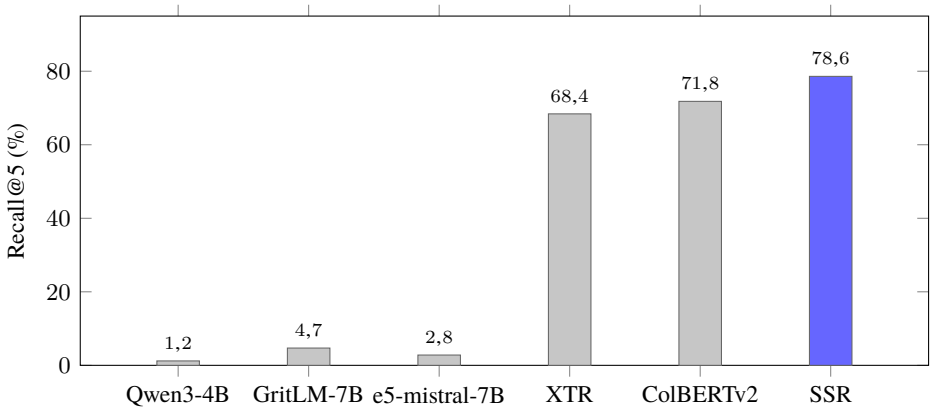
Bảng 5.5: Kết quả trên bộ LIMIT theo bài báo.

Phương pháp	Recall@5	Recall@10	Recall@100
Qwen3-4B (SVR)	1,2	4,6	6,3
GritLM-7B (SVR)	4,7	6,1	19,4
e5-mistral-7B (SVR)	2,8	5,2	10,9
ColBERTv2 (MVR)	71,8	82,6	94,4
XTR (MVR)	68,4	79,9	93,2
SSR	**78,6**	**87,3**	**98,1**

Bảng 5.5 cho thấy rõ hạn chế của biểu diễn single-vector: các mô hình SVR mạnh nhất cũng chỉ đạt Recall@5 dưới 5%, trong khi SSR đạt 78,6. Đây là lý do MVR vẫn đáng theo đuổi dù chi phí cao hơn.

5.2.4. Hiệu quả tính toán và tài nguyên hệ thống

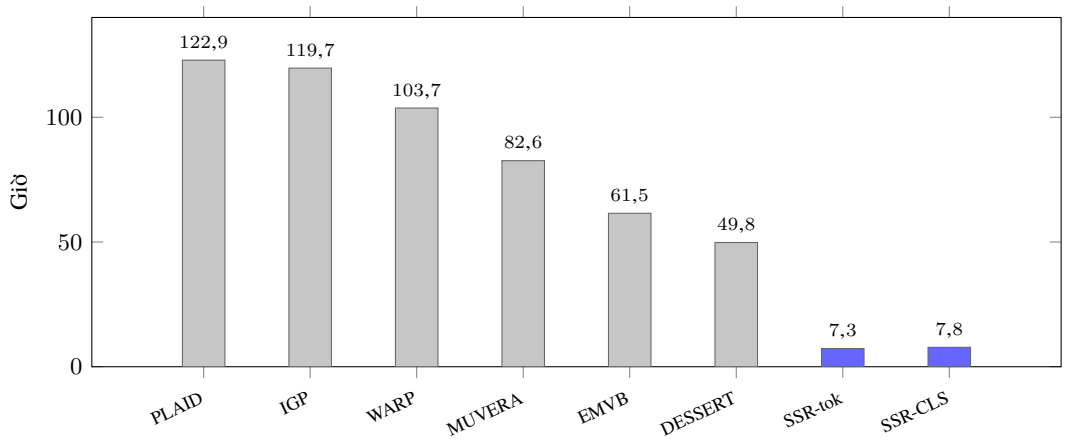
Theo Bảng 5.6, SSR giảm thời gian lập chỉ mục xuống còn 7,3–7,8 giờ, tức khoảng 1/15 so với PLAID, đồng thời đạt MRR@10 cao nhất trên CPU. Về tài nguyên (Bảng 4.1), bộ nhớ đỉnh giảm từ 274,2 GB (ColBERTv2/PLAID) xuống 34,6 GB (SSR-tok), chỉ mục gọn hơn (18,5 GB so với 22,1 GB) và có thể cập nhật bằng cách nối thêm. Về huấn luyện, ColBERTv2 cần khoảng 24,2 giờ còn SSR-tok giảm khoảng 53%. Hình 5.4 tái hiện Hình 3 của bài: thời gian toàn pipeline và ảnh hưởng của quy mô kho dữ liệu (khi mở rộng từ 0,5 triệu lên 10 triệu tài liệu, SSR-tok chỉ giảm 5,3% chất lượng so với 8,9% của ColBERTv2 theo bản v3).



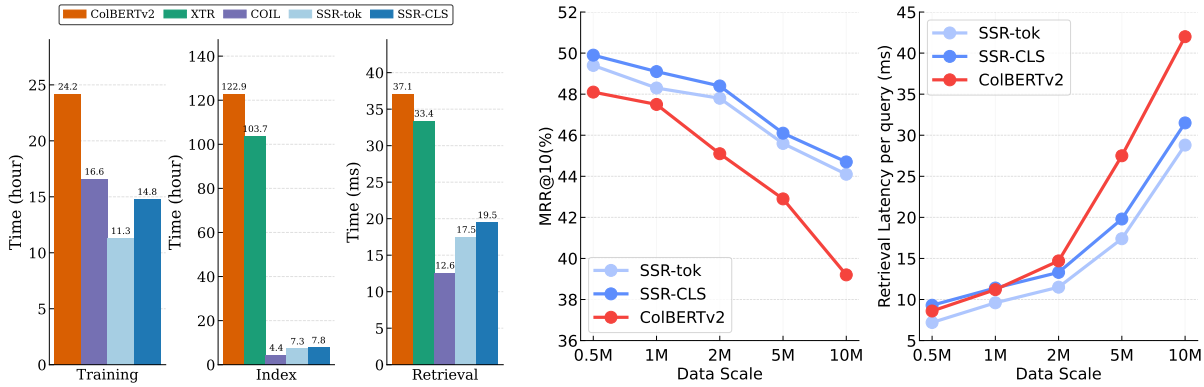
Hình 5.2: Recall@5 trên benchmark LIMIT theo bài báo (vẽ lại từ Bảng 5.5): khoảng cách rất lớn giữa các mô hình SVR và MVR.

Bảng 5.6: So sánh hiệu quả trên CPU theo bài báo (MS MARCO passage; chỉ mục tính bằng giờ, truy hồi tính bằng ms/truy vấn).

Phương pháp	MRR@10	Lập chỉ mục (h)	Truy hồi (ms)
PLAID	39,6	122,9	156,3
WARP	38,7	103,7	129,7
EMVB	39,8	61,5	93,4
DESSERT	37,2	49,8	75,6
MUVERA	39,7	82,6	94,7
IGP	39,1	119,7	113,2
SSR-tok	39,5	**7,3**	**49,1**
SSR-CLS	**40,2**	7,8	57,3



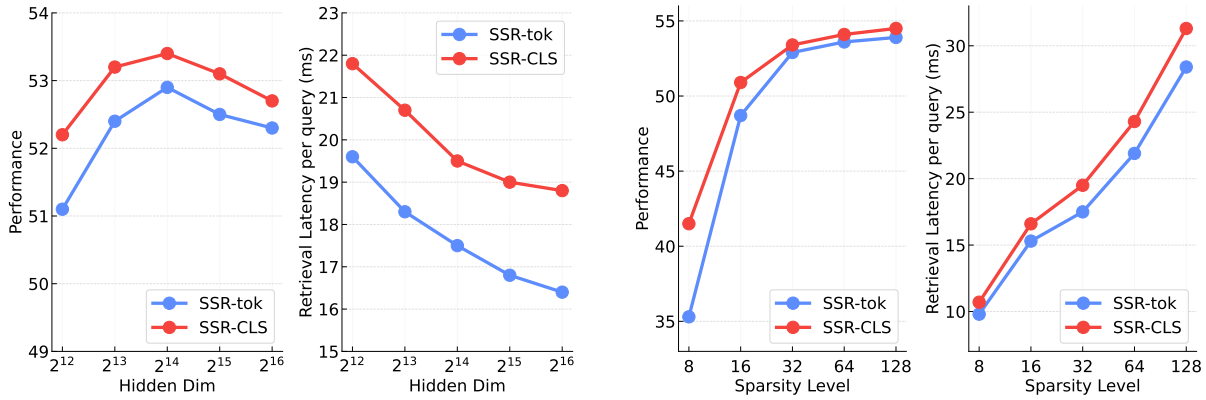
Hình 5.3: Thời gian lập chỉ mục MS MARCO trên CPU của các phương pháp MVR theo bài báo (vẽ lại từ Bảng 5.6).



(a) End-to-end analysis. **(b)** Effect of data scale.

Hình 5.4: Trái: thời gian huấn luyện, lập chỉ mục và truy hồi trong toàn bộ pipeline. Phải: chất lượng và độ trễ so với ColBERTv2 ở các quy mô dữ liệu khác nhau. Nguồn: Hình 3 của bài báo [1].

5.2.5. Phân tích siêu tham số h và K



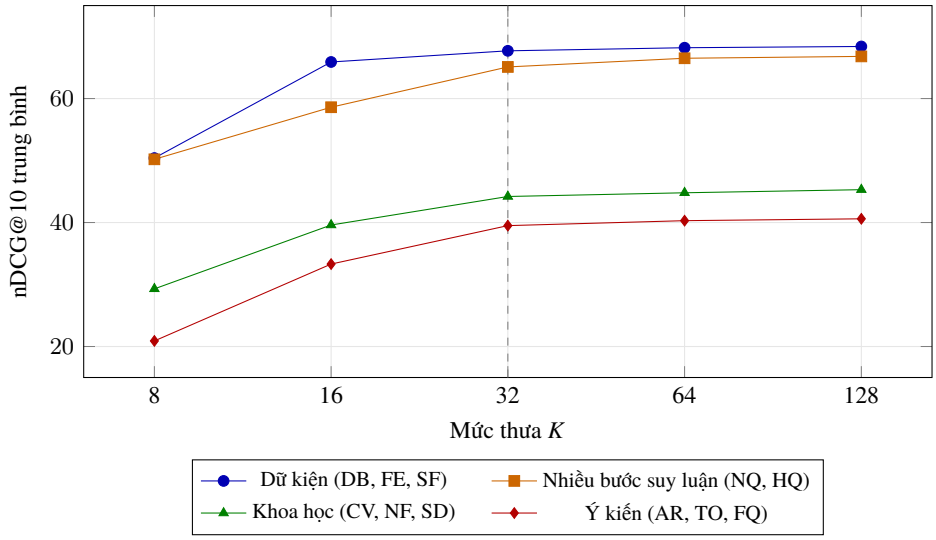
(a) Hidden dimension. **(b)** Sparsity level.

Hình 5.5: Trái: ảnh hưởng của hidden dimension h (từ 2^{12} đến 2^{16}). Phải: ảnh hưởng của mức thưa K (từ 8 đến 128). Nguồn: Hình 4 của bài báo [1].

Đường cong chất lượng theo h có dạng chữ U ngược với đỉnh ở $h = 2^{14}$: khi h quá lớn so với K, kích hoạt bị phân mảnh khiến các token liên quan ít chia sẻ neuron với nhau, làm giảm số điểm giao hữu ích trong inverted index. Chất lượng theo K có điểm gãy ở $K = 32$: khi $K < 16$ chất lượng giảm mạnh, còn khi K lớn thì độ trễ tăng mà lợi ích không đáng kể. Bảng 5.7 và Hình 5.6 cho thấy $K = 32$ ổn định ở hầu hết các miền, còn nhóm tác vụ suy luận nhiều bước hưởng lợi rõ từ $K = 64$.

Bảng 5.7: nDCG@10 theo mức thưa K và theo nhóm miền dữ liệu (theo bài báo).

Miền	$K = 8$	$K = 16$	$K = 32$	$K = 64$	$K = 128$
Dữ kiện (DB, FE, SF)	50,4	65,9	67,7	68,2	68,4
Nhiều bước suy luận (NQ, HQ)	50,2	58,6	65,1	66,5	66,8
Khoa học (CV, NF, SD)	29,3	39,6	44,2	44,8	45,3
Ý kiến (AR, TO, FQ)	20,9	33,3	39,5	40,3	40,6



Hình 5.6: Ảnh hưởng của K lên chất lượng theo từng nhóm miền dữ liệu (vẽ lại từ Bảng 5.7); đường đứt đánh dấu $K = 32$.

K thích ứng theo độ dài truy vấn ($\leq$ 3 token dùng $K = 16$; 4–7 dùng $K = 32$; $\geq$ 8 dùng $K = 64$) đạt chất lượng tương đương $K = 64$ cố định nhưng độ trễ chỉ 16,3 ms. Việc đổi K lúc suy luận khả thi nhờ số hạng Multi-TopK ở Chương 3.

Bảng 5.8: K cố định và K thích ứng theo độ dài truy vấn (theo bài báo).

Thiết lập	Cố định 16	Cố định 32	Cố định 64	Thích ứng
nDCG@10 trung bình	50,5	52,9	53,1	53,0
Độ trễ (ms)	16,4	17,5	19,9	16,3

5.2.6. Các phiên bản của bài báo và vấn đề độ đo

Khi tính lại các bảng, nhóm thấy một điểm cần chú ý khi đọc Bảng 5.2:

- **arXiv v3 (03/06/2026) và bản PMLR:** chú thích Bảng 1 ghi "MS MARCO báo cáo bằng MRR@10"; nhưng số của SSR-tok (45,2) thực ra là nDCG@10, còn ColBERTv2 (39,8), PLAID (39,7) là MRR@10. Hai độ đo này không so được với nhau (mục 5.1.2): trên cùng một hệ thống, nDCG@10 thường cao hơn MRR@10 vài điểm.
- **arXiv v4 (28/09/2026):** đổi MS MARCO sang nDCG@10 ở mọi bảng và cập nhật baseline: ColBERTv2 39,8 → 44,2; PLAID 39,7 → 44,1; COIL 35,3 → 40,9; ColBERT 36,0 → 41,1. Lợi thế in-domain của SSR giảm từ +5,4 xuống +1,0. V4 cũng sửa trung bình của ColBERT (41,5 → 44,8) và dòng XTR ở Bảng 9 (47,6 → 45,0), đúng hai mâu thuẫn tính lại được từ v3; thí nghiệm theo quy mô dữ liệu đổi từ "SSR giảm 5,3% so với 8,9%" thành "4,2% so với 4,7%".
- **Bản chính thức PMLR** đổi nhãn sang nDCG@10 nhưng giữ số baseline của v3, nên so sánh lệch chỉ số vẫn nằm trong bản đã xuất bản. Bảng 15 ở cả hai bản vẫn ghi MRR@10: SSR-tok 39,5, PLAID 39,6, EMVB 39,8, tức theo chính bảng này SSR chỉ ngang PLAID trên MS MARCO (Bảng 5.6).

5.3. Cài đặt, kiểm chứng và tái lập của nhóm

Đây là phần thực hành của nhóm, được trình bày tách biệt với số liệu của bài báo. Mã nguồn (bản repo đã sửa, patch, script đánh giá, kết quả và log) nằm tại https://github.com/duykhanhxx03/ssr-repro-and-optimization.

5.3.1. Môi trường và dữ liệu

Bảng 5.9: Môi trường thực nghiệm của nhóm.

Hạng mục	**Giá trị**
Phần cứng	CPU Intel i5-12400F (6 nhân / 12 luồng), RAM 32 GB, GPU NVIDIA RTX 5060 8 GB
Hệ điều hành	Windows 11
Phần mềm	Python 3.13, PyTorch 2.11 (CUDA 12.8), transformers 5.3, sentence-transformers 5.3, PyLate 1.6 (ColBERT + fast-plaid), numba 0.61, numpy, scipy
Mã nguồn	`Y-Research-SBU/SSR` commit `c64bc05` (26/06/2026; không có commit nào mới hơn tới ngày 12/10/2026)
Checkpoint	`SSR-tok` tải từ HuggingFace ngày 11/10/2026 (tác giả đưa lên ngày 05/10/2026)
Baseline	`colbert-ir/colbertv2.0` chạy qua PyLate; BM25 cài bằng numpy theo công thức Lucene ($k_1 = 0{,}9$, $b = 0{,}4$)

Các tập BEIR tải theo định dạng MTEB bằng script `prepare_mteb_eval.py` của repo, có bổ sung các split train/dev từ bộ BEIR gốc để phục vụ kiểm tra ở mục 5.3.6. Mọi số đo thời gian đều chạy khi máy không có tác vụ khác, sau giai đoạn khởi động.

5.3.2. Cấu trúc mã nguồn công bố

Bảng 5.10: Các tập dữ liệu nhóm đã dùng.

Tập dữ liệu	Lĩnh vực	Số tài liệu	Query (test)	Split khác đã dùng
NFCorpus	y sinh	3.633	323	dev 324, train 2.590
SciFact	khoa học	5.183	300	train 809
FiQA-2018	tài chính	57.638	648	dev 500, train 1.000 (mẫu)
Quora	câu hỏi trùng	522.931	10.000	dev 5.000
MS MARCO passage	web, in-domain	8.841.823	dev-small 6.980; DL19 43	không

Listing 5.1: Các thành phần chính của repository SSR và vị trí tương ứng trong báo cáo.

```
SSR/
  prepare_msmarco.py         # tai va dong goi MS MARCO thanh triplet
  prepare_mteb_eval.py       # tai cac tap BEIR theo dinh dang MTEB
  ssr/
    model.py                 # lop SSR = ColBERT (PyLate) + SparseAutoencoder
    sparse_autoencoder.py    # SAE TopK, AuxK, annealing     -> muc 3.2
    losses.py                # recon, aux, contrastive, CE   -> muc 3.3-3.5
    dataset.py, train.py     # nap triplet MS MARCO, huan luyen
    retrieval/
      inverted_index.py      # dung chi muc nguoc, cham diem  -> muc 4.2-4.3
      exact_retriever.py     # duong "exact"
      retriever.py           # SparseMaxSimRetriever (SSR++)  -> muc 4.4
      eval_mteb.py           # CLI: --variant ssr|ssr-cls, --mode exact|pruned
      metrics.py, mteb_io.py # nDCG, MRR; doc qrels           -> muc 5.1.2
  tests/
```

Lệnh đánh giá chuẩn của repo (sau khi đã sửa) là:

```
python -m ssr.retrieval.eval_mteb --task retrieval --variant ssr \
   --model-path ckpt/SSR-tok --dataset nfcorpus --data-dir data/mteb \
   --mode exact --score-device index --split test
```

5.3.3. Kiểm tra pipeline đánh giá bằng baseline

Trước khi đo SSR, nhóm kiểm tra rằng pipeline đánh giá (đọc dữ liệu, tính nDCG có trọng số, mã hóa, MaxSim) cho đúng số đã công bố với các mô hình đã biết. Bảng 5.11 cho thấy sai lệch dưới 0,6 điểm, nằm trong biên độ khác biệt phiên bản thư viện. Nhờ đó, mọi khác biệt sau này của SSR so với bài báo không thể quy cho pipeline.

Bảng 5.11: Tái lập số đã công bố của các baseline bằng pipeline của nhóm.

Mô hình	Tập / độ đo	Đã công bố	Nhóm đo
ColBERTv2	SciFact nDCG@10	69,3	69,16
ColBERTv2	NFCorpus nDCG@10	33,8	34,40
ColBERTv2	FiQA nDCG@10	35,6	35,80
ColBERTv2	Quora nDCG@10	85,2	85,67
BM25 (Lucene)	MS MARCO dev-small MRR@10	18,4 (Anserini)	18,38
BM25 (Lucene)	TREC DL19 nDCG@10	50,6 (Anserini)	50,72

5.3.4. Rà soát mã nguồn: các lỗi phát hiện và cách sửa

Nhóm đối chiếu từng hàm trong repo với công thức tương ứng của bài. Bảng 5.12 tổng hợp các điểm sai lệch. Các lỗi 2, 6, 7, 9 đã được tác giả xác nhận trong hai issue trên GitHub (mục 3.7); các lỗi 1, 3, 4, 5, 8 là phát hiện của nhóm, chưa được nêu trong các issue. Toàn bộ bản sửa được đóng gói thành một patch (16 file, áp dụng sạch lên commit `c64bc05`) kèm 7 unit test. Chi tiết từng lỗi cùng đoạn mã trước và sau khi sửa được trình bày trong Phụ lục A.

Bảng 5.12: Các sai lệch giữa mã nguồn công bố và bài báo.

#	Lỗi	Ảnh hưởng	Theo bài	Trạng thái
1	Chế độ `exact` trên inverted index tính $\sum_i \max_u q_i^{(u)} \max_j d_j^{(u)}$ thay vì MaxSim; đường Python, numba và hàm "tham chiếu" sai giống nhau	NFCorpus 8,33 / SciFact 7,46 → 43,97 / 87,83	Công thức (4)	Đã sửa
2	`load_ssr` dựng SAE ngẫu nhiên 3.072 latent thay vì nạp checkpoint; bỏ qua `config_sentence_transformers.json` (độ dài tài liệu 180 thay vì 512, bật query expansion)	Không nạp được mô hình	không ghi	Đã sửa
3	Đường `e2e-index` bỏ mất `latent_remap`: query và chỉ mục dùng hai hệ chỉ số neuron khác nhau	0,012 → 0,440	không ghi	Đã sửa
4	Bước thô SSR++ không theo công thức (12); tham số 8 / 32.768 / 512 thay vì $K_{\text{coarse}} = 4$, 2.000 ứng viên, block 64	SSR++ khác bài	(11)–(12), D.1	Đã sửa
5	nDCG tính với nhãn nhị phân, bỏ mức liên quan	≈1 điểm trên NFCorpus	D.1	Đã sửa
6	Thiếu số hạng $\frac{1}{8}\mathcal{L}_{\text{recon}}(4k)$ trong loss	Huấn luyện	(7)	Đã sửa
7	Tham số huấn luyện mặc định lệch Bảng 6 ($\gamma = 0$ tắt $\mathcal{L}_{\text{CE}}$, lr $2\cdot10^{-5}$, batch 256, 1 epoch, không warmup, $k_{\text{aux}} = 1024$)	Huấn luyện	Bảng 6	Thêm preset
8	Config SAE của checkpoint có khóa lạ; dựng chỉ mục tạo một object Python cho mỗi posting (tràn RAM từ ~50k tài liệu)	Không chạy được corpus lớn	không ghi	Đã sửa
9	Data loader chỉ lấy 1 hard negative mỗi query; tác giả dùng 32 và gom theo query (chỉ nói trong issue #1)	Huấn luyện	Không ghi	Sửa theo issue
10	`token_skip_mode` (bỏ stopword) chỉ có trong config checkpoint, mã nguồn không có	Mã hóa	Không ghi	Chưa sửa

Trong các lỗi trên, lỗi 1 ảnh hưởng lớn nhất đến kết quả và chưa được nêu trong các issue trên GitHub. Bộ cộng dồn của chế độ `exact` lấy giá trị lớn nhất theo từng neuron rồi mới gộp, nên điểm của cặp (token query i, tài liệu D) trở thành $\max_u q_i^{(u)} \mu_{D,u}$ thay vì MaxSim theo (2.2), đúng như phân tích ở mục 2.3. Nhóm viết lại kernel chấm điểm để cộng theo từng cặp (tài liệu, token) rồi mới lấy max theo token và cộng theo token query, và kiểm chứng bằng ba cách tính độc lập (MaxSim trên vector dense bằng PyTorch, một cài đặt bằng `scipy.sparse`, bản "unblocked" của repo đã sửa cùng lỗi); cả ba trùng nhau tới 10^{-6} trên mọi cặp query và tài liệu của NFCorpus và SciFact. Vì lệnh đánh giá mặc định của repo đi qua đúng đường này, con số 0,108 mà hai nhóm trong issue #1 thu được sau khi tự sửa lỗi nạp SAE nhiều khả năng cũng bị kéo xuống bởi lỗi này.

5.3.5. Kết quả chạy lại trên split test

Bảng 5.13 so sánh nDCG@10 (có trọng số, chuẩn BEIR) của checkpoint công bố khi chạy bằng mã gốc (chỉ sửa lỗi nạp SAE) và bằng mã đã sửa, cùng với số trong bài và ColBERTv2 đo lại.

Bảng 5.13: nDCG@10 trên split test (nhóm đo). "Repo gốc" = checkpoint công bố + mã gốc (đã sửa lỗi nạp SAE); "đã sửa" = MaxSim đúng công thức (4); SSR++ theo cấu hình Phụ lục D.1.

Tập (số tài liệu)	Bài báo	Repo gốc	SSR-tok đã sửa	SSR++ đã sửa	ColBERTv2
NFCorpus (3,6k)	38,8	8,3	43,97	43,98	34,40
SciFact (5,2k)	72,9	7,5	87,83	87,83	69,16
FiQA (57,6k)	39,4	không chạy	41,41	36,86	35,80
Quora (523k)	88,3	không chạy	85,42	không chạy	85,67

Listing 5.2: Trích log chạy NFCorpus bằng mã đã sửa (`logs/A_fixed_nfcorpus.log`).

```
2026-10-11 20:39:34,255 [INFO] Metrics for nfcorpus: mrr@[] ndcg@[10]
2026-10-11 20:40:02,045 [INFO]   ndcg@10: 0.4397
```

Một số nhận xét từ Bảng 5.13:

- Sau khi sửa, mã chạy đúng công thức và SSR vượt ColBERTv2 trên split test của ba tập đầu, đúng chiều với tuyên bố của bài.

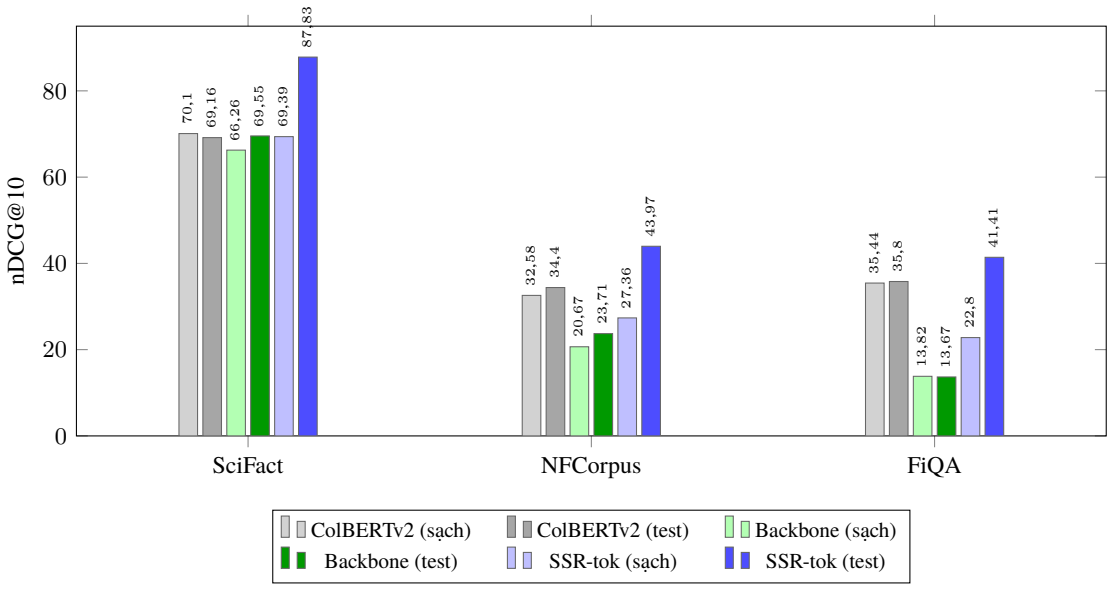
- Tuy nhiên checkpoint công bố cho số *cao hơn* bài báo trên 3/4 tập (SciFact 87,8 so với 72,9), nên đây không phải mô hình đã dùng cho Bảng 1. SciFact 87,8 với một mô hình cỡ BERT-base cao hơn mọi mô hình 7–8B trong Bảng 2 của chính bài, một dấu hiệu bất thường cần kiểm tra thêm (mục 5.3.6).
- SSR++ không mất điểm trên NFCorpus và SciFact chỉ vì 2.000 ứng viên gần như phủ hết corpus (3,6k đến 5,2k tài liệu). Trên FiQA (57,6k) nó mất 4,6 điểm, trái với Bảng 4.3.

5.3.6. Kiểm tra theo split: điểm chỉ vọt lên ở split test

Bài báo tuyên bố đánh giá zero-shot: mô hình chỉ học MS MARCO. Khi đó chất lượng trên split train/dev và split test của *cùng một* tập BEIR phải xấp xỉ nhau, vì chúng được lấy mẫu từ cùng phân phối. Nhóm đo ba hệ thống trên cùng corpus: ColBERTv2 (đối chứng), SSR-tok (có SAE), và "backbone dense" là chính BERT của checkpoint SSR nhưng bỏ qua SAE, chấm MaxSim trên vector 768 chiều.

Bảng 5.14: nDCG@10 trên split sạch (train/dev) và split test của cùng một tập (nhóm đo). Quora dùng quy ước BEIR bỏ tài liệu trùng id với query.

Tập (split sạch → test)	Mô hình	Split sạch	Test	Test − sạch
SciFact (train → test)	ColBERTv2	70,10	69,16	−0,9
	Backbone dense (không SAE)	66,26	69,55	+3,3
	SSR-tok	69,39	87,83	+18,4
NFCorpus (dev → test)	ColBERTv2	32,58	34,40	+1,8
	Backbone dense	20,67	23,71	+3,0
	SSR-tok	27,36	43,97	+16,6
FiQA (dev → test)	ColBERTv2	35,44	35,80	+0,4
	Backbone dense	13,82	13,67	−0,2
	SSR-tok	22,80	41,41	+18,6
Quora (dev → test)	ColBERTv2	85,25	85,67	+0,4
	SSR-tok	82,16	85,42	+3,3



Hình 5.7: Chất lượng trên split sạch và split test (nhóm đo). Chỉ SSR-tok (có SAE) có bước nhảy lớn ở split test.

Kết quả (Bảng 5.14, Hình 5.7) cho thấy:

- ColBERTv2 phẳng giữa các split (chênh dưới 2 điểm), đúng như kỳ vọng của đánh giá zero-shot. Backbone BERT của chính checkpoint SSR cũng phẳng.
- SSR-tok vọt lên 16–19 điểm chỉ ở split test của SciFact, NFCorpus, FiQA. Các split sạch khác (NFCorpus

train 2.590 query: 28,76; FiQA train 1.000 query: 20,91) cho cùng kết quả như dev. Với vài trăm đến vài nghìn query mỗi split, chênh lệch cỡ này khó có thể do ngẫu nhiên.

- Vì backbone không có hiện tượng này, phần bất thường nằm ở tầng SAE: trên split sạch SAE chỉ thêm 3–9 điểm so với backbone, nhưng thêm 18–28 điểm trên split test. Hiện tượng này phù hợp với khả năng SAE từng được huấn luyện hoặc chọn mô hình khi có mặt dữ liệu test của BEIR. Mã huấn luyện công bố chỉ đọc MS MARCO, nên nhóm chỉ ghi nhận hiện tượng, không kết luận về nguyên nhân.
- Do đó, trên dữ liệu sạch, checkpoint công bố ngang hoặc kém ColBERTv2 (SciFact $-0{,}7$; NFCorpus $-5{,}2$; FiQA $-12{,}6$; Quora $-3{,}1$).

5.3.7. Đánh giá in-domain trên MS MARCO

MS MARCO là dữ liệu huấn luyện của SSR. Truy hồi toàn bộ 8,84 triệu đoạn văn cần chỉ mục mức token khoảng 70–190 GB RAM, vượt khả năng của máy, nên nhóm dùng giao thức re-ranking chuẩn: BM25 lấy top-100 cho mỗi query, rồi mọi mô hình chấm lại cùng tập ứng viên bằng MaxSim chính xác. Kết quả bị giới hạn bởi recall của BM25 (66,8% query dev-small có đoạn liên quan trong top-100) nên không so trực tiếp với số full retrieval trong bài; so sánh *giữa các mô hình* là công bằng.

Bảng 5.15: Re-ranking BM25 top-100 trên MS MARCO (nhóm đo). Pipeline SSR được kiểm tra lại: sparse code của query trùng 235/235 dòng với hàm của repo, điểm khớp kernel exact tới $1{,}4 \cdot 10^{-6}$.

Hệ thống	dev-small MRR@10	dev-small nDCG@10	TREC DL19 nDCG@10
BM25 (không re-rank)	18,38	22,83	50,72
ColBERTv2	**35,86**	**40,92**	**72,72**
SSR-tok (checkpoint công bố)	23,18	28,29	57,96
Backbone dense của SSR (không SAE)	15,94	20,34	47,43

Ngay trên dữ liệu in-domain, checkpoint công bố kém ColBERTv2 12,7 điểm MRR@10 và 14,8 điểm nDCG@10 trên DL19, cùng chiều với các split sạch ở mục 5.3.6 và trái với Bảng 5.2 (SSR-tok 45,2 so với ColBERTv2 39,8). Tầng SAE vẫn có ích so với chính backbone của nó (+7,2 MRR@10), nhưng backbone này là một retriever dense yếu, thấp hơn cả BM25, cho thấy BERT đã bị thay đổi nhiều trong quá trình huấn luyện (tác giả xác nhận không đóng băng BERT).

5.3.8. So sánh hiệu năng với ColBERTv2 + PLAID

Các tuyên bố về hiệu năng được kiểm tra trên FiQA (57.638 tài liệu, khoảng 9,8 triệu token) với PLAID của PyLate/fast-plaid (nén 2 bit, chạy GPU; bản CPU không dựng xong chỉ mục sau 50 phút nên không đưa vào). Độ trễ là một query mỗi lần, top-100, sau khởi động. Bảng 5.16 cũng đưa trước các số của bản tối ưu ở mục 5.4 để tiện so sánh.

Bảng 5.16: Hiệu năng trên FiQA (nhóm đo). Thời gian mã hóa corpus không tính vào "dựng chỉ mục" (ColBERTv2 ≈260 s, SSR ≈420 s trên GPU).

Hệ thống	Dựng index	Kích thước	ms/query (p95)	Thiết bị	nDCG test	nDCG dev
ColBERTv2 + PLAID	32,6 s	0,59 GiB	44,7 (51)	GPU	35,73	34,93
SSR-tok, repo đã sửa (exact)	164 s	2,41 GiB	1.635	CPU 1 luồng	41,41	22,80
SSR-tok, `fast_hyb` (exact, nhóm)	2,9 s	3,96 GiB	59,0 (93)	CPU 12 luồng	41,41	22,80
SSR-tok, LB-rerank t=8/1000 (nhóm)	2,9 s	3,96 GiB	36,7 (57)	CPU 12 luồng	41,41	22,81
SSR-tok, exact GPU (nhóm)	không có	3,2 GB VRAM	55,9 (60)	GPU	41,41	22,80
SSR++ cấu hình bài (bản tăng tốc)	2,9 s	3,96 GiB	11,7 (17)	CPU 12 luồng	36,86	18,11

Listing 5.3: Trích log đo PLAID (`logs/plaid_bench.log`).

```
[plaid] index cuda: build 32.6s, 603 MiB, encode 284s
[plaid] test cuda: {'ndcg@10': 35.727, 'mean_ms': 44.70, 'p50_ms': 44.44, 'p95_ms': 51.15, 'n_queries': 648}
[plaid] dev cuda:  {'ndcg@10': 34.934, 'mean_ms': 44.18, 'p50_ms': 44.07, 'p95_ms': 50.74, 'n_queries': 500}
```

Bảng 5.17: Đối chiếu từng tuyên bố về hiệu năng của bài báo với số nhóm đo trên FiQA.

Bài báo nói	Đo được (FiQA)	Đánh giá
Dựng chỉ mục nhanh hơn 15× vì bỏ K-means (7,3 h so với 122,9 h trên MS MARCO)	K-means + nén của PLAID chỉ mất 32,6 s trên GPU; phần tốn thời gian là mã hóa. Tổng: PLAID ≈290 s, SSR ≈420 s	Không thấy ở quy mô này
PLAID phải dựng lại khi thêm tài liệu, SSR chỉ cần thêm posting	fast-plaid có sẵn `add_documents`, `remove_documents`, `update_documents`	Phóng đại
Truy hồi nhanh gần 2× PLAID	SSR exact 56–59 ms so với PLAID 44,7 ms; chỉ SSR++ nhanh hơn rõ (11,7 ms) nhưng mất 4,6 điểm	Đúng khi chấp nhận mất chất lượng
Chỉ mục nhỏ hơn PLAID (18,5 GB so với 22,1 GB)	SSR 2,4–4,0 GiB so với PLAID 0,59 GiB, lớn hơn 4–7 lần	Ngược chiều
SSR++ hầu như không mất chất lượng (Bảng 5)	FiQA test 41,41 → 36,86; dev 22,80 → 18,11	Không đúng trên corpus lớn
$h = 16.384$ neuron (Bảng 6)	Checkpoint chỉ có 8.192 neuron hoạt động; nửa còn lại trọng số 0, bias -10	Khác cấu hình

Về chất lượng trên cùng FiQA: trên split dev (sạch) SSR-tok đạt 22,80 so với 34,93 của PLAID; SSR chỉ vượt PLAID trên split test. Nguyên nhân SSR mã hóa chậm hơn ColBERTv2 là tài liệu dài tới 512 token (ColBERTv2: 300) và tầng SAE sinh hàng 16.384 chiều cho mỗi token; với batch 32 các tensor này tràn 8 GB VRAM và chậm đi khoảng 8 lần, nên nhóm mã hóa với batch 16.

5.3.9. Thông tin từ trả lời của tác giả trên GitHub

Bảng 5.18 đối chiếu các thông tin bài không ghi mà tác giả cung cấp trong hai issue [19] với những gì nhóm kiểm tra được.

Bảng 5.18: Thông tin từ trả lời của tác giả và đối chiếu của nhóm.

Tác giả nói	Đối chiếu
Token `[MASK]` mở rộng query bị loại trước MaxSim; nhiệt độ $\tau = 0{,}3$	Khớp config checkpoint (`do_query_expansion: false`) và cách chạy của nhóm
BERT không đóng băng; lr 10^{-3} trong Bảng 6 "không ổn định", khoảng 10^{-6} tốt hơn	Bảng 6 ghi 10^{-3}; nhóm thêm preset `train_issue1_setup.sh`
32 hard negative mỗi query, 2 query × 32 triplet mỗi batch 64; warmup 20.000 bước ≈4% của 2 epoch	Nhóm cài đặt (`--negatives-per-query`, `ssr/samplers.py`), chạy thử được với batch 16
Số hạng $\frac{1}{8}\mathcal{L}_{\text{recon}}(4k)$ có dùng khi huấn luyện, chỉ sót khi công bố	Nhóm thêm lại
Trên NFCorpus hơn 70% neuron được kích hoạt (đo bằng script thăm dò)	Checkpoint công bố: 7.453/16.384 neuron (45,5%); không thể vượt 50% vì chỉ 8.192 neuron hoạt động

5.3.10. Giới hạn của thực nghiệm

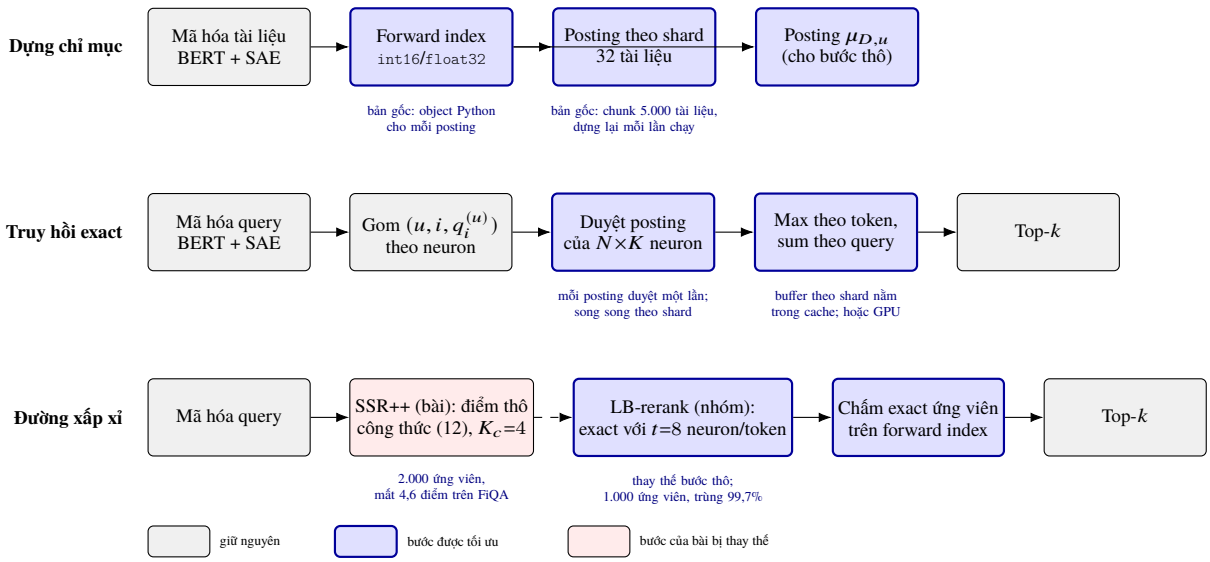
- MS MARCO chỉ kiểm bằng re-ranking; full retrieval cần ≥70 GB RAM cho chỉ mục mức token.
- Chưa huấn luyện lại đầy đủ (khoảng 500k bước GPU) và chưa kiểm biến thể Llama-Embed-8B (cần hơn 8 GB VRAM); chưa kiểm SSR-CLS vì không có checkpoint.
- Độ trễ đo trên RTX 5060 / i5-12400F, không so trực tiếp được với A100 trong bài; chỉ so sánh tương đối giữa các hệ thống trên cùng máy.
- Mọi kết luận về chất lượng áp dụng cho checkpoint công bố trên HuggingFace, không phải mô hình chưa công bố đã dùng cho Bảng 1.

5.4. Cải tiến mã nguồn: tối ưu thuật toán truy hồi

Mục tiêu là giảm độ trễ của phép truy hồi exact (công thức (2.2)) mà giữ nguyên kết quả; mọi so sánh chất lượng của phương án xấp xỉ dùng split sạch. Mốc so sánh là bản exact đã sửa của repo (một luồng, corpus chia thành từng chunk 5.000 tài liệu). Toàn bộ mã nằm trong thư mục `opt/` của repo nhóm.

5.4.1. Vị trí tối ưu trong pipeline truy hồi

Hình 5.8 mô tả pipeline xử lý một query của SSR sau khi sửa và đánh dấu các bước được tối ưu. Phần mã hóa query (BERT + SAE) và phần chọn top-k giữ nguyên; nhóm thay đổi ba chỗ: cách dựng và lưu chỉ mục, hai bước chấm điểm trên chỉ mục (duyệt posting và gộp max/sum), và bước sinh ứng viên của đường xấp xỉ. Bảng 5.19 liệt kê từng bước với cách làm của bản gốc và của bản tối ưu.



Hình 5.8: Pipeline truy hồi của SSR và vị trí các tối ưu của nhóm. Hàng trên: dựng chỉ mục; hàng giữa: truy hồi exact (công thức (4)); hàng dưới: đường xấp xỉ, trong đó bước thô của SSR++ được thay bằng LB-rerank.

Bảng 5.19: Từng bước của pipeline: cách làm của bản gốc (repo đã sửa) và của bản tối ưu.

Bước	Bản gốc (repo đã sửa)	Bản tối ưu	Kết quả (FiQA)
Dựng chỉ mục	Mỗi posting là một object Python, sắp xếp bằng Python; corpus chia thành từng chunk 5.000 tài liệu và dựng lại ở mỗi lần chạy	Forward index `fw_lat int16[n_tok,K]`, `fw_val float32[n_tok,K]`; posting đếm và điền bằng numba, lưu shard-major (khóa = shard$\times h$ + neuron)	4,8 s thay vì ≈280 s; RAM đỉnh 7,5 GB thay vì 24,5 GB
Mã hóa query, gom theo neuron	Giữ nguyên	Giữ nguyên	–
Duyệt posting	Với mỗi token query và mỗi neuron của nó, duyệt lại posting của neuron đó (một neuron chung cho m token bị duyệt m lần); một luồng	Mỗi neuron của query duyệt một lần, cập nhật mọi token chứa nó (cập nhật dạng vector khi ≥ 3 token, vô hướng khi ít hơn); các shard chạy song song (`prange`)	Chiếm phần lớn mức tăng tốc 5× (1 luồng) đến 30× (12 luồng)
Max theo token, sum theo query	Buffer $N \times 5000$ tài liệu, không nằm trong cache; gộp sau khi duyệt xong cả chunk	Gộp ngay trong shard trên buffer $n_{\text{tok}} \times N$ cỡ vài KB; bản GPU thay hai bước này bằng SpMM (CSR) + `segment_reduce`	GPU: 56 ms/query
Top-k	`argpartition`	Giữ nguyên	–
Sinh ứng viên (đường xấp xỉ)	Điểm thô theo công thức (12) trên posting μ, $K_{\text{coarse}} = 4$, 2.000 ứng viên	LB-rerank: chấm exact với query cắt còn $t = 8$ neuron mỗi token (cận dưới của công thức (4)), 1.000 ứng viên, rồi chấm exact trên forward index	nDCG 22,81 so với 18,11 của SSR++; 0,67 thời gian exact

Phần tối ưu không đụng tới mô hình hay cách biểu diễn: biểu diễn thưa và công thức chấm điểm giữ nguyên, nên mọi kết quả xếp hạng của đường exact trùng khớp với bản gốc. Các thay đổi nằm ở cấu trúc dữ liệu của chỉ mục và thứ tự thực hiện các vòng lặp trong hai bước chấm điểm, cộng thêm một cách chọn ứng viên khác cho đường xấp xỉ.

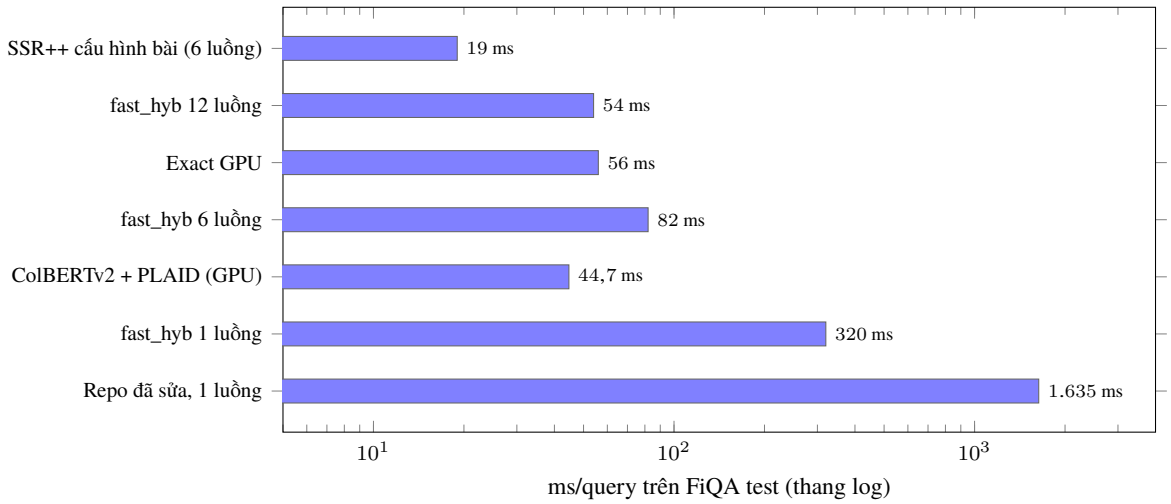
5.4.2. Kernel exact mới `fast_hyb`

Phân tích bản gốc cho thấy ba điểm tốn thời gian: (i) buffer điểm theo (token, tài liệu) có kích thước $N \times 5000$ không nằm trong cache; (ii) mỗi neuron của query được duyệt lại một lần cho mỗi token query chứa nó; (iii) chỉ chạy một luồng. Kernel mới (`opt/ssr_opt/fast_exact.py`) thay đổi như sau:

- **Chia nhỏ theo tài liệu:** corpus chia thành các shard 32 tài liệu liên tiếp; posting được lưu shard-major (khóa = shard $\times h$ + neuron) nên buffer nháp của một shard chỉ vài KB và nằm gọn trong cache L1/L2. Các shard xử lý song song bằng numba `prange`.
- **Duyệt mỗi posting một lần:** trong một shard, với mỗi neuron u của query, duyệt posting (d, t, v) của u đúng một lần và cập nhật *mọi* token query chứa u (bản gốc duyệt lại cho từng token).
- **Forward index:** lưu thêm `fw_lat int16[n_tok, K]` và `fw_val float32[n_tok, K]` để chấm chính xác một tập tài liệu bất kỳ (dùng cho bước tinh của SSR++ và LB-rerank). Chỉ mục FiQA dựng trong 4,8 s với đỉnh RAM 7,5 GB, so với 174 s + 107 s và 24,5 GB của bản gốc.
- **Giữ thứ tự cộng:** các tích được cộng theo thứ tự neuron tăng dần như kernel gốc, nên điểm số giống hệt từng bit; test `opt/tests/test_fast_exact.py` kiểm tra chênh lệch bằng 0 trên mọi tài liệu và mọi query của NFCorpus và SciFact.

Bảng 5.20: Độ trễ của các kernel exact (nhóm đo; kết quả xếp hạng giống hệt bản gốc).

Tập	Phương pháp	Luồng	nDCG@10	ms/query (p95)	Tăng tốc
FiQA test	Repo đã sửa (mặc định)	1	41,41	1.635	1×
FiQA test	`fast_hyb`	1	41,41	320 (497)	5,1×
FiQA test	`fast_hyb`	6	41,41	82 (127)	19,9×
FiQA test	`fast_hyb`	12	41,41	54 (82)	30,0×
FiQA test	Exact trên GPU (CSR + segment max)	GPU	41,41	56 (60)	29,3×
FiQA test	SSR++ cấu hình bài (bản tăng tốc)	6	36,86	19 (27)	86×
SciFact test	Repo đã sửa → `fast_hyb`	1 → 12	87,83	354 → 12,8	28×
NFCorpus test	Repo đã sửa → `fast_hyb`	1 → 12	43,99	43,6 → 2,5	17×



Hình 5.9: Độ trễ truy vấn trên FiQA (nhóm đo). Mọi phương án của SSR trừ SSR++ đều cho kết quả xếp hạng giống hệt bản gốc.

5.4.3. Kernel exact trên GPU

Nhóm cài thêm một bản exact trên GPU (`opt/ssr_opt/gpu_exact.py`): chỉ mục lưu dạng CSR (chỉ số int64), tích giữa query và token tính bằng SpMM rồi lấy max theo tài liệu bằng `torch.segment_reduce`. Hai điểm cần chú ý khi chạy trên Windows: phải mã hóa toàn bộ query trước rồi giải phóng mô hình để tránh tràn VRAM sang bộ nhớ chia sẻ (chậm 8 lần), và `segment_reduce` nhanh hơn nhiều so với `scatter_reduce` với tensor mở rộng. Kết quả 56 ms/query, ngang bản CPU 12 luồng, sai số số học cỡ 10^{-6} (float32) và top-100 trùng hoàn toàn khi xét đến hòa điểm.

5.4.4. Phương án xấp xỉ mới: LB-rerank

SSR++ chấm điểm thô bằng (4.4), tức bỏ $K - K_{\text{coarse}} = 28$ neuron của mỗi token query, nên trên FiQA mất nhiều ứng viên (mục 4.4.3). Nhóm đề xuất *LB-rerank*: thay vì điểm thô trên posting μ, xếp ứng viên bằng MaxSim *chính xác* của phiên bản query chỉ giữ t neuron lớn nhất của mỗi token. Vì mọi giá trị không âm, điểm này là một *cận dưới* của (2.2) nhưng vẫn ràng buộc các neuron thuộc cùng một token tài liệu, nên xếp hạng gần với điểm thật hơn nhiều. Sau đó chấm exact (đủ K) trên 1.000 ứng viên bằng forward index.

Bảng 5.21: Các phương án xấp xỉ trên FiQA dev (nhóm đo, 6 luồng; exact = 82 ms, nDCG@10 22,80).

Phương án	nDCG@10	Recall@100	Trùng top-10 với exact	Thời gian / exact
SSR++ cấu hình bài (K_{coarse} 4, 2.000 ứng viên)	18,11	33,10	0,59	0,22
SSR++ K_{coarse} 4, 10.000 ứng viên	22,25	48,74	0,88	0,73
SSR++ K_{coarse} 32, 2.000 ứng viên	16,30	29,83	0,51	0,37
LB-rerank t = 8, 1.000 ứng viên	**22,81**	**50,72**	**0,997**	0,67
LB-rerank t = 4, 1.000 ứng viên	22,40	46,23	0,91	0,52

LB-rerank $t = 8$ gần như không mất điểm (top-10 trùng 99,7% với exact) ở 0,67 lần thời gian; FiQA train (1.000 query) cho cùng xu hướng (exact 20,91; LB-rerank 20,91; SSR++ cấu hình bài 17,90). Tăng K_{coarse} lên 32 (dùng đủ neuron trong bước thô) thậm chí còn làm SSR++ kém hơn, cho thấy vấn đề của bước thô nằm ở việc mất ràng buộc "cùng một token tài liệu" chứ không phải thiếu neuron.

5.4.5. Các hướng đã thử nhưng không hiệu quả

- **Dừng sớm an toàn (safe early termination) bằng cận trên** (công thức (4.4) với đủ K neuron, kết hợp chặn Cauchy-Schwarz theo chuẩn token): đúng về toán và cho top-k giống hệt, nhưng cận quá lỏng nên chậm hơn quét toàn bộ 2–4 lần. Bảng 5.22 giải thích: ngưỡng để lọt top-100 chỉ khoảng 0,22–0,32 cho mỗi token query, trong khi độ lỏng của cận trên là 0,34–0,51, nên 97–99% tài liệu vẫn nằm trên ngưỡng và không thể bị bỏ qua.
- **Pruning theo block (block-max)** như bài mô tả: cận theo block còn lỏng hơn cận theo tài liệu nên không triển khai tiếp. Vì vậy nhóm cho rằng cơ chế "bỏ qua block theo U_B" của bài đóng góp ít vào tốc độ.
- **Gộp nhiều query một lần:** tăng thông lượng tối đa khoảng 1,2×.

Bảng 5.22: Độ chặt của cận trên dùng để pruning (nhóm đo trên 40 query mỗi tập). θ: ngưỡng top-100 trung bình mỗi token query; "UB $\geq \theta$": tỉ lệ tài liệu có cận trên vượt ngưỡng (không bỏ qua được).

Tập	θ mỗi token	Cận	UB $\geq \theta$	Độ lỏng mỗi token
NFCorpus test	0,22	posting μ (đủ K)	97,6%	0,34
NFCorpus test	0,22	t = 8 neuron	94,7%	0,23
SciFact train	0,32	posting μ (đủ K)	99,3%	0,51
SciFact train	0,32	t = 8 neuron	98,3%	0,37

5.4.6. Tổng kết phần cải tiến

Phần cải tiến mã nguồn của nhóm có hai mặt. Về tính đúng, nhóm sửa mười điểm sai lệch so với bài báo (Bảng 5.12), đóng gói thành patch và unit test, để mã nguồn công bố chạy đúng công thức và chạy được trên corpus lớn. Về tốc độ, kernel `fast_hyb` nhanh hơn 19–30 lần với kết quả giống hệt từng bit, bản GPU tương đương, chỉ mục dựng trong vài giây thay vì vài phút với một phần ba bộ nhớ, và phép xấp xỉ LB-rerank thay cho bước thô của SSR++ với chất lượng gần như không đổi. Với các cải tiến này, SSR exact trên CPU 12 luồng đạt độ trễ cùng cỡ với PLAID chạy trên GPU (54–59 ms so với 44,7 ms) trên corpus 57k tài liệu.

5.5. Thảo luận

5.5.1. Điểm mạnh của phương pháp SSR

- **Ý tưởng đơn giản, tận dụng cấu trúc dữ liệu sẵn có:** biểu diễn thưa TopK biến mỗi neuron thành một "từ", cho phép dùng inverted index, cấu trúc đã được tối ưu nhiều năm cho tìm kiếm từ khóa, mà vẫn giữ tương tác giữa các token của ColBERT. Không cần K-means, codebook hay giải nén residual.
- **Chỉ mục cập nhật được:** thêm tài liệu chỉ là thêm posting, không phụ thuộc vào codebook đã học.
- **Tiềm năng tốc độ:** với kernel phù hợp (mục 5.4), truy hồi exact đạt 54–59 ms/query trên CPU cho 57k tài liệu và dựng chỉ mục trong vài giây; biểu diễn thưa cũng cho phép xấp xỉ có kiểm soát (LB-rerank) gần như không mất chất lượng.
- **Có phân tích lý thuyết:** Phụ lục A của bài cho chặn sai số giữa MaxSim thưa và MaxSim dense (mục 2.5); phương pháp tổng quát hóa cho cả BERT lẫn backbone LLM 8B.

5.5.2. Hạn chế của bài báo và mô hình

- **Không tái lập được từ mã nguồn công bố:** mã chấm điểm sai công thức (4), hàm nạp checkpoint tạo SAE ngẫu nhiên, thiếu số hạng loss, data loader khác mô tả của tác giả. Chính tác giả xác nhận mã công bố không phải mã tạo ra số trong bài và cách huấn luyện thật (lr, số negative, nhiệt độ) khác Bảng 6.
- **Checkpoint công bố có dấu hiệu tiếp xúc dữ liệu test:** điểm chỉ vọt lên ở split test của BEIR, trong khi trên split sạch và trên MS MARCO in-domain nó kém ColBERTv2 từ 0,7 đến 12,7 điểm. Checkpoint cũng chỉ dùng 8.192/16.384 neuron, khác cấu hình trong bài.
- **So sánh lệch chỉ số trên MS MARCO** ở bản v3 và bản PMLR; sau khi v4 sửa, lợi thế in-domain chỉ còn +1,0.
- **Các tuyên bố hiệu năng chỉ đúng một phần:** ở quy mô 57k tài liệu, thời gian dựng chỉ mục bị chi phối bởi mã hóa (SAE làm chậm hơn ColBERTv2), chỉ mục lớn hơn PLAID 4–7 lần, và SSR++ mất 4,6 điểm trên corpus lớn. Hiệu quả còn phụ thuộc phần cứng, bố cục chỉ mục và độ dài posting list, điều tác giả cũng thừa nhận trong phần tác động.
- **Những điểm chưa rõ trong bài:** cận lý thuyết chưa được kiểm chứng định lượng qua ϵ và δ; cách kết hợp điểm `[CLS]` với điểm token không được mô tả; hai siêu tham số h và K phải dò và K tối ưu thay đổi theo miền; trên miền tranh luận và quan điểm, SSR còn kém SPLADE, và về độ trễ thuần SPLADE vẫn nhanh hơn một chút (16,3 ms) và đơn giản hơn.
- **Chi phí bộ nhớ khi mã hóa:** tầng SAE sinh hàng 16.384 chiều cho mỗi token, dễ tràn VRAM với batch lớn hoặc tài liệu dài.

5.5.3. Khả năng ứng dụng thực tiễn

- **Tìm kiếm ngữ nghĩa trên hạ tầng từ khóa:** vì chỉ mục của SSR là inverted index thông thường, có thể nhúng vào các máy tìm kiếm sẵn có (Lucene/Elasticsearch) dưới dạng pseudo-token, tương tự cách SPLADE được triển khai, mà vẫn giữ tương tác giữa các token.
- **Kho tài liệu thay đổi liên tục:** tin tức, hỗ trợ khách hàng, tài liệu nội bộ, nơi việc dựng lại codebook K-means theo chu kỳ là gánh nặng.
- **Hệ thống RAG:** bước truy hồi cho mô hình ngôn ngữ lớn cần chất lượng tốt hơn BM25 nhưng độ trễ thấp và chi phí dự đoán được; kết hợp với kernel `fast_hyb` hoặc LB-rerank của nhóm, SSR đạt vài chục ms/query trên CPU.

- **Điều kiện áp dụng:** cần một checkpoint được huấn luyện minh bạch; với checkpoint hiện tại, chất lượng trên dữ liệu sạch chưa bằng ColBERTv2.

5.5.4. Tác động xã hội

Theo phần tác động của bài báo, truy hồi nhanh hơn có thể khuếch đại thiên lệch xếp hạng hiện có, tăng tốc tìm kiếm hoặc phát tán nội dung có hại, hay hỗ trợ các mục đích giám sát nếu thiếu cơ chế bảo vệ. Những rủi ro đó không chỉ xuất hiện ở SSR, nhưng hiệu quả truy hồi cao hơn giúp các quy trình thiên lệch mở rộng dễ dàng hơn. Triển khai có trách nhiệm cần kèm kiểm tra thiên lệch, giám sát kết quả xếp hạng ở các tác vụ tiếp theo và biện pháp bảo vệ phù hợp với lĩnh vực sử dụng. Ở góc độ nghiên cứu, kinh nghiệm tái lập của nhóm cho thấy việc công bố mã nguồn và checkpoint là chưa đủ; cộng đồng cần kiểm chứng độc lập trước khi dựa vào các con số công bố.

5.6. Kết luận và hướng phát triển

5.6.1. Kết luận

Multi-Vector Retrieval giữ được độ chi tiết mức token và nhờ đó đạt độ chính xác cao, nhưng cách tổ chức hiện đại dựa trên K-means clustering đã biến lợi thế này thành gánh nặng: lập chỉ mục tốn hàng trăm giờ, bộ nhớ đỉnh lên tới hàng trăm GB, và mỗi khi thêm tài liệu lại phải xây lại chỉ mục. Báo cáo đã trình bày phương pháp SSR như một lời giải thay thế: chiếu token lên một không gian nhiều chiều nhưng cực kỳ thưa bằng TopK Sparse Autoencoder, biến mỗi neuron hoạt động thành một pseudo-token và cho phép tái sử dụng trực tiếp inverted index. Nhóm đã trình bày ba phần chính của phương pháp: cơ sở lý thuyết về sai lệch bị chặn giữa điểm thưa và điểm dense (Chương 2), hàm mục tiêu huấn luyện lai gồm tái tạo, chống neuron chết, contrastive giữa token và contrastive giữa truy vấn và tài liệu (Chương 3), cùng cấu trúc inverted index mức neuron và thuật toán pruning từ thô đến tinh SSR++ (Chương 4). Về lý thuyết, đây là hướng đi hợp lý, kết hợp được ưu điểm của lexical retrieval và Multi-Vector Retrieval.

Về thực nghiệm, nhóm đã chạy lại mã nguồn và checkpoint công bố, phát hiện và sửa mười điểm sai lệch (trong đó lỗi công thức chấm điểm trên inverted index chưa từng được báo cáo), đóng gói thành patch có test, và kiểm chứng kết quả bằng nhiều cách tính độc lập cùng baseline đối chứng. Kết quả cho thấy mã công bố không tái lập được số trong bài; checkpoint công bố chỉ vượt ColBERTv2 trên đúng split test của BEIR và kém ColBERTv2 trên split sạch cũng như trên MS MARCO in-domain; một số tuyên bố hiệu năng không đúng ở quy mô nhóm đo được; và bản arXiv v4 đã tự sửa một lỗi so sánh lệch chỉ số.

Về cải tiến mã nguồn, nhóm thiết kế lại kernel chấm điểm exact (nhanh hơn 19–30 lần trên FiQA, kết quả giống hệt từng bit, dựng chỉ mục 4,8 s thay vì khoảng 280 s), cài bản GPU, và đề xuất phép xấp xỉ LB-rerank gần như không mất chất lượng, tốt hơn SSR++ của bài trên corpus lớn.

Qua đề tài, nhóm rút ra rằng kết quả công bố cần được kiểm chứng bằng baseline đối chứng trên cùng pipeline, bằng split sạch bên cạnh split test, và bằng cách đối chiếu mã nguồn với từng công thức trong bài. Đây là kinh nghiệm chính nhóm thu được từ môn học, bên cạnh kiến thức về inverted index, MaxSim và biểu diễn thưa.

5.6.2. Hướng phát triển

- **Huấn luyện lại minh bạch:** dùng mã đã sửa (đủ số hạng $4k$, 32 negative gom theo query, $\tau = 0{,}3$) trên GPU lớn hơn để tạo checkpoint có thể kiểm chứng; đánh giá trên split sạch và full retrieval MS MARCO; đo ϵ và δ của SAE để kiểm chứng định lượng cận lý thuyết.
- **Giảm kích thước chỉ mục:** quantization giá trị posting (8 bit), nén chỉ số token, hoặc giảm độ dài tài liệu về 300 token như ColBERTv2 để thu hẹp khoảng cách 4–7 lần với PLAID.
- **Pruning an toàn hiệu quả hơn:** cận trên hiện quá lỏng; có thể học cận theo cặp neuron hoặc dùng cấu trúc "impact-ordered" kết hợp với LB-rerank để có dừng sớm có bảo đảm.

- **Sparsity thích ứng:** chọn số neuron cho từng token truy vấn theo phân bố kích hoạt (giữ neuron u nếu $z^{(u)} \geq \tau \cdot \max_v z^{(v)}$) thay vì theo độ dài truy vấn; nhờ Multi-TopK không cần huấn luyện lại.
- **Tích hợp tín hiệu toàn cục:** kết hợp SSR-CLS hoặc trộn với BM25 để bù cho token thưa trên tài liệu dài và trên các miền tranh luận, quan điểm.
- **Mở rộng backbone:** kiểm chứng biến thể Llama-Embed-8B của bài trên phần cứng đủ VRAM, và thử SAE trên các mô hình embedding đa ngôn ngữ cho tiếng Việt.

TÀI LIỆU THAM KHẢO

[1] L. Guo, Y. Wang, T. Wen, A. Feng, S. Jegelka, C. You, "No More K-means: Single-Stage Sparse Coding for Efficient Multi-Vector Retrieval," *Proc. 43rd International Conference on Machine Learning (ICML)*, PMLR 306, 2026. arXiv:2605.30120 (v3 03/06/2026; v4 28/09/2026).

[2] O. Khattab, M. Zaharia, "ColBERT: Efficient and Effective Passage Search via Contextualized Late Interaction over BERT," *Proc. SIGIR*, 2020, pp. 39–48.

[3] K. Santhanam, O. Khattab, J. Saad-Falcon, C. Potts, M. Zaharia, "ColBERTv2: Effective and Efficient Retrieval via Lightweight Late Interaction," *Proc. NAACL*, 2022, pp. 3715–3734.

[4] K. Santhanam, O. Khattab, C. Potts, M. Zaharia, "PLAID: An Efficient Engine for Late Interaction Retrieval," *Proc. CIKM*, 2022, pp. 1747–1756.

[5] L. Gao, Z. Dai, J. Callan, "COIL: Revisit Exact Lexical Match in Information Retrieval with Contextualized Inverted List," *Proc. NAACL*, 2021.

[6] J. Lee, Z. Dai, S. M. K. Duddu, T. Lei, I. Naim, M.-W. Chang, V. Zhao, "Rethinking the Role of Token Retrieval in Multi-Vector Retrieval," *Advances in Neural Information Processing Systems*, vol. 36, 2023.

[7] J. L. Scheerer, M. Zaharia, C. Potts, G. Alonso, O. Khattab, "WARP: An Efficient Engine for Multi-Vector Retrieval," *Proc. SIGIR*, 2025, pp. 2504–2512.

[8] F. M. Nardini, C. Rulli, R. Venturini, "Efficient Multi-Vector Dense Retrieval with Bit Vectors," *Proc. ECIR*, 2024.

[9] T. Formal, B. Piwowarski, S. Clinchant, "SPLADE: Sparse Lexical and Expansion Model for First Stage Ranking," *Proc. SIGIR*, 2021; T. Formal, C. Lassance, B. Piwowarski, S. Clinchant, "SPLADE v2," arXiv:2109.10086, 2021; C. Lassance, H. Déjean, T. Formal, S. Clinchant, "SPLADE-v3: New Baselines for SPLADE," arXiv:2403.06789, 2024.

[10] L. Gao, T. Dupré la Tour, H. Tillman, G. Goh, R. Troll, A. Radford, I. Sutskever, J. Leike, J. Wu, "Scaling and Evaluating Sparse Autoencoders," arXiv:2406.04093, 2024 (ICLR 2025).

[11] H. Lee, A. Battle, R. Raina, A. Y. Ng, "Efficient Sparse Coding Algorithms," *Advances in Neural Information Processing Systems*, vol. 19, 2006.

[12] N. Thakur, N. Reimers, A. Rücklé, A. Srivastava, I. Gurevych, "BEIR: A Heterogeneous Benchmark for Zero-shot Evaluation of Information Retrieval Models," *NeurIPS Datasets and Benchmarks*, 2021.

[13] T. Nguyen, M. Rosenberg, X. Song, J. Gao, S. Tiwary, R. Majumder, L. Deng, "MS MARCO: A Human Generated MAchine Reading COmprehension Dataset," *CoCo@NeurIPS*, 2016.

[14] O. Weller, M. Boratko, I. Naim, J. Lee, "On the Theoretical Limitations of Embedding-Based Retrieval," arXiv:2508.21038, 2025.

[15] S. E. Robertson, S. Walker, S. Jones, M. Hancock-Beaulieu, M. Gatford, "Okapi at TREC-3," *Proc. TREC-3*, 1995.

[16] K. Järvelin, J. Kekäläinen, "Cumulated Gain-based Evaluation of IR Techniques," *ACM Transactions on Information Systems*, vol. 20, no. 4, pp. 422–446, 2002.

[17] P. Yang, H. Fang, J. Lin, "Anserini: Enabling the Use of Lucene for Information Retrieval Research," *Proc. SIGIR*, 2017, pp. 1253–1256.

[18] A. Chaffin, R. Sourty, "PyLate: Flexible Training and Retrieval for Late Interaction Models," LightOn, 2024. https://github.com/lightonai/pylate.

[19] Mã nguồn chính thức của SSR: https://github.com/Y-Research-SBU/SSR (commit c64bc05, 26/06/2026); Issue #1 "Difficulty reproducing BEIR numbers from Table 1" (24/07/2026) và Issue #2 "Is the 4k reconstruction loss missing from the training objective?" (04/08/2026).

[20] Checkpoint SSR-tok: https://huggingface.co/Y-Research-Group/SSR (đưa lên 05/10/2026).

[21] Bản chính thức trên PMLR: https://proceedings.mlr.press/v306/guo26x.html.

[22] Mã nguồn, patch, script đánh giá và kết quả của nhóm: https://github.com/duykhanhxx03/ssr-repro-and-optimization.

PHỤ LỤC A. CHI TIẾT CÁC LỖI TRONG MÃ NGUỒN VÀ BẢN SỬA

Phụ lục này trình bày chi tiết các lỗi đã tóm tắt trong Bảng 5.12 (mục 5.3.4), kèm đoạn mã trước và sau khi sửa. Toàn bộ bản sửa nằm trong patch `ssr_paper_fixes.patch` của repo nhóm, áp dụng được lên commit `c64bc05` của mã nguồn gốc bằng `git apply`.

A.1. Lỗi 2: nạp checkpoint tạo SAE ngẫu nhiên

Hàm `load_ssr` gọi `SSR(model_name_or_path=...)`; vì PyLate chỉ giữ các module `Transformer`/`Dense` khi nạp, module `SparseAutoencoder` bị bỏ và một SAE mới với $4 \times 768 = 3072$ latent được khởi tạo ngẫu nhiên. Mọi đánh giá bằng lệnh mặc định của repo do đó chạy trên một SAE chưa huấn luyện. Một người dùng trong issue #1 cũng phát hiện lỗi này độc lập.

Listing A.1: `ssr/retrieval/eval_mteb.py`: trước (trên) và sau khi sửa (dưới).

```python
def load_ssr(model_path: Path, device: str) -> SSR:
    model = SSR(model_name_or_path=str(model_path), device=device)   # SAE ngau nhien
    model.eval()
    return model

def load_ssr(model_path: Path, device: str, token_scope=None) -> SSR:
    # SSR(model_name_or_path=...) would attach a randomly initialised SAE.
    return load_ssr_checkpoint(str(model_path), device=device,
                               token_scope=token_scope)
```

Hàm mới `load_ssr_checkpoint` trong `ssr/model.py` đọc `modules.json`, nạp SAE từ thư mục con `1_SparseAutoencoder` (lọc các khóa config lạ), và khôi phục các thiết lập ColBERT lưu trong `config_sentence_transformers.json`: tiền tố `[Q]`/`[D]`, độ dài query 192 và tài liệu 512, `do_query_expansion = false`, danh sách skiplist.

A.2. Lỗi 1: công thức chấm điểm trên inverted index

Lỗi này ảnh hưởng lớn nhất đến kết quả và chưa được nêu trong các issue. Bộ cộng dồn của chế độ `exact` làm việc theo từng neuron: với mỗi neuron u của token query i, nó lấy giá trị lớn nhất của u trong mỗi tài liệu rồi cập nhật ô (i, D) bằng phép `maximum`:

Listing A.2: `inverted_index.py` gốc: bộ cộng dồn lấy max theo từng neuron.

```python
for q_entries, s, e in segments:          # moi segment = mot neuron u
    docs = doc_idx[s:e]; vals = values[s:e]
    out_docs, max_vals = segment_max_by_doc(docs, vals)   # mu_{D,u}
    for qt, v_q in q_entries:             # cac token query chua u
        scaled = max_vals * np.float32(v_q)
        row = local[qt]
        row[out_docs] = np.maximum(row[out_docs], scaled)  # max, khong phai cong
```

Kết quả là điểm của cặp (token query i, tài liệu D) bằng $\max_u q_i^{(u)} \mu_{D,u}$: mỗi neuron được phép chọn một token tài liệu khác nhau, và chỉ neuron lớn nhất được giữ. So với (2.2), phép tính này vừa đánh mất phép cộng trên các neuron chung, vừa không ràng buộc các neuron phải thuộc cùng một token tài liệu. Kết quả đo được: NFCorpus 8,33 và SciFact 7,46 nDCG@10, thấp hơn cả BM25.

Bản sửa (`ssr/retrieval/exact_index_maxsim.py`, kernel numba) cộng dồn theo đúng hàng (tài liệu, token tài liệu), rồi mới lấy max theo token trong tài liệu và cộng theo token query:

Listing A.3: Kernel exact mới (rút gọn): cộng theo (doc, token) rồi mới max theo token.

```
for qt in range(n_qt):                      # moi token query
    gen_state[0] += 1; gen = gen_state[0]; n_rows = 0
    for k in range(qt_offsets[qt], qt_offsets[qt + 1]):   # moi neuron u cua qt
        li = seg_rows[k]; vq = seg_vq[k]
        for p in range(offsets[li], offsets[li + 1]):     # posting (doc, token, val)
            d = doc_idx[p]
            r = row_start[d] + token_idx[p]               # hang = (doc, token)
            if row_gen[r] != gen:                         # lan dau cham hang nay
                row_gen[r] = gen; acc[r] = 0.0
                touched_rows[n_rows] = r; n_rows += 1
            acc[r] += vq * values[p]                      # SUM tren neuron chung
    for i in range(n_rows):                               # MAX theo token trong doc
        r = touched_rows[i]; d = row_doc[r]
        if doc_gen[d] != gen:
            doc_gen[d] = gen; doc_best[d] = acc[r]
            touched_docs[n_docs] = d; n_docs += 1
        elif acc[r] > doc_best[d]:
            doc_best[d] = acc[r]
    for i in range(n_docs):                               # SUM theo token query
        d = touched_docs[i]; doc_total[d] += doc_best[d]
```

Vì mọi giá trị sau TopK+ReLU đều không âm, các token tài liệu không chung neuron nào với token query đóng góp 0 và có thể bỏ qua mà không đổi kết quả max. Bản sửa được kiểm chứng bằng ba cách độc lập: (i) so với MaxSim tính trên vector thưa dạng dense bằng PyTorch (`reference_maxsim_dense`); (ii) so với một cài đặt bằng `scipy.sparse` viết riêng (`verify_scipy.py`); (iii) so với bản "unblocked" của repo đã sửa cùng lỗi (`crosscheck.py`). Cả ba trùng nhau tới 10^{-6} trên mọi cặp query và tài liệu của NFCorpus và SciFact.

A.3. Lỗi 3 và 4: ánh xạ neuron và bước thô SSR++

Chỉ mục nén các neuron thực sự xuất hiện trong corpus thành chỉ số liên tiếp (`latent_remap`); đường `e2e-index` quên áp ánh xạ này lên query nên phần giao giữa neuron query và neuron tài liệu chỉ còn khoảng 2% (nDCG@10 0,012). Sau khi ánh xạ query bằng `remap_sparse_embeddings`, kết quả là 0,440, trùng với đường chuẩn.

Bước thô của SSR++ trong repo dùng một cơ chế pruning theo block khác với (4.4) và tham số mặc định ($K_{\text{coarse}} = 8$, 32.768 ứng viên, block 512) không khớp Phụ lục D.1. Nhóm cài đặt lại đúng (4.4) trên posting $\mu_{D,u}$ (được dựng từ posting token bằng một phép sắp xếp), lấy top-2.000 toàn cục trên mọi block, rồi chấm tinh bằng kernel exact với mask `allowed`:

Listing A.4: Kernel chấm điểm thô theo công thức (12).

```
@njit(cache=True)
def _coarse_totals_kernel(mu_offsets, mu_doc, mu_val, seg_rows, seg_vq, doc_total):
    for k in range(seg_rows.shape[0]):        # (neuron u, q_i^{(u)}) voi u trong top-Kcoarse
        li = seg_rows[k]; vq = seg_vq[k]
        for p in range(mu_offsets[li], mu_offsets[li + 1]):
            doc_total[mu_doc[p]] += vq * mu_val[p]     # q_i^{(u)} * mu_{D,u}
```

A.4. Lỗi 5–7 và 9: độ đo và huấn luyện

Hàm nDCG của repo đọc qrels thành tập nhị phân, làm mất mức liên quan 2 của NFCorpus và 2–3 của TREC DL; nhóm sửa để giữ `Dict[str, Dict[str, int]]` và dùng độ lợi $2^{\text{rel}} - 1$. Trong `losses.py`, `recons_4k` được tính trong forward của SAE nhưng không đưa vào loss; nhóm thêm thành phần `reconstruction_4k` với hệ số 0,125 (issue #2 xác nhận tác giả có dùng số hạng này). Nhóm cũng thêm `--warmup-steps`, `--negatives-per-query` và một `QueryGroupedBatchSampler` gom các triplet của cùng query vào một batch như tác giả mô tả:

Listing A.5: `ssr/samplers.py`: gom triplet cùng query vào một batch.

```python
class QueryGroupedBatchSampler(DefaultBatchSampler):
    def __init__(self, dataset, batch_size, drop_last, ...):
        super().__init__(dataset, batch_size=batch_size, drop_last=drop_last, ...)
        queries = dataset["query"]; groups = []
        for i, q in enumerate(queries):      # cac hang lien tiep cung query = 1 nhom
            if i > 0 and q == queries[i - 1]: groups[-1].append(i)
            else: groups.append([i])
        self.groups = groups

    def _batches(self):
        g = torch.Generator(); g.manual_seed(int(self.seed) + int(self.epoch))
        order = torch.randperm(len(self.groups), generator=g).tolist()
        batches, current = [], []
        for gi in order:                     # xep nguyen nhom vao batch
            group = self.groups[gi]
            if current and len(current) + len(group) > self.batch_size:
                batches.append(current); current = []
            current = current + group
        if current: batches.append(current)
        return batches
```

Hai preset `scripts/train_table6.sh` (đúng Bảng 6) và `scripts/train_issue1_setup.sh` (cấu hình tác giả nói trong issue: lr 10^{-6}, 32 negative gom theo query, $\tau = 0{,}3$, warmup 20.000) được thêm vào. Đường huấn luyện đã chạy thử trọn vẹn trên một tập MS MARCO nhỏ (batch 16, 8 negative mỗi query; log `logs/train_smoke.log` cho thấy `reconstruction_4k` và warmup lr hoạt động, checkpoint nạp lại được).

A.5. Lỗi 8: dựng chỉ mục với corpus lớn

Hàm dựng chỉ mục gốc tạo một object Python cho mỗi posting rồi sắp xếp bằng Python, nên với FiQA (258 triệu posting) tràn RAM 32 GB. Nhóm thay bằng hai lượt đếm rồi điền trên mảng numpy (và kernel numba trong bản tối ưu), đồng thời lọc các khóa lạ trong `config.json` của SAE để checkpoint nạp được. Sau sửa, chỉ mục FiQA dựng trong 164 s bằng đường của repo và 2,9–4,8 s bằng đường tối ưu (Bảng 5.16).

PHỤ LỤC B. HƯỚNG DẪN CHẠY LẠI THÍ NGHIỆM

Các lệnh dưới đây chạy trong thư mục gốc của repo nhóm (https://github.com/duykhanhxx03/ssr-repro-and-optimization) sau khi đã cài môi trường (Bảng 5.9); đường dẫn và tham số đầy đủ ghi trong `README.md` của repo.

Listing B.1: Cài đặt môi trường, dữ liệu và checkpoint.

```
python -m venv venv && source venv/bin/activate
pip install torch --index-url https://download.pytorch.org/whl/cu128
pip install -e "SSR[eval,data]" pylate sentence-transformers transformers numba scipy pytest
# BEIR (dinh dang MTEB) vao data/mteb/<dataset>/ ; them split train/dev tu BEIR goc
python SSR/prepare_mteb_eval.py --datasets nfcorpus scifact fiqa quora
# Checkpoint SSR-tok
huggingface-cli download Y-Research-Group/SSR --include "SSR-tok/*" --local-dir ckpt
```

Listing B.2: Các thí nghiệm chính (số hiệu theo mục tương ứng trong Chương 5).

```
# 5.3.5  Danh gia checkpoint cong bo bang ma da sua (MaxSim theo cong thuc (4))
python -m ssr.retrieval.eval_mteb --task retrieval --variant ssr \
    --model-path ckpt/SSR-tok --dataset nfcorpus --data-dir data/mteb \
    --mode exact --score-device index --split test        # nDCG@10 = 0.4397

# 5.3.6  Kiem tra theo split va cac mo hinh doi chung
python run_eval.py --ckpt ckpt/SSR-tok --token-scope non-cls -- \
    --task retrieval --variant ssr --dataset scifact --data-dir data/mteb --split train
python eval_dense_colbert.py lightonai/colbertv2.0 scifact train,test
python eval_backbone_dense.py scifact train,test
python quora_ssr.py --dataset quora --splits test,dev --ignore-identical-ids

# 5.3.4  Ba cach tinh doc lap xac nhan loi cham diem
python crosscheck.py --ckpt ckpt/SSR-tok --data-dir data/mteb --dataset nfcorpus \
    --corpus-npz cache/A/nfcorpus/corpus_32k_test.npz
python verify_scipy.py nfcorpus cache/A/nfcorpus/corpus_32k_test.npz
python verify_bug.py   nfcorpus cache/A/nfcorpus/corpus_32k_test.npz

# 5.3.7  MS MARCO: BM25 top-100 roi re-rank
python msmarco_rerank.py bm25
python msmarco_rerank.py rerank --models ssr colbertv2 backbone

# 5.3.8 / 5.4  Hieu nang: PLAID va kernel toi uu tren FiQA
python plaid_bench.py
python verify_fast_hyb.py nfcorpus test cache/A/nfcorpus/corpus_32k_test.npz
bash opt/run_all.sh fiqa_fast fiqa_sweep
python opt/bench.py --dataset fiqa --split dev --methods fast_hyb fast_ssrpp --threads 6

# Unit test
python -m pytest SSR/tests -q
python opt/tests/test_fast_exact.py
```